



# KSIPF: an effective noise filtering oversampling method based on k-means and iterative-partitioning filter

Pengfei Sun<sup>1</sup> · Zhiping Wang<sup>1</sup> · Liyan Jia<sup>1</sup> · Xiaoxi Wang<sup>2</sup>

Accepted: 16 February 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

## Abstract

The Synthetic Minority Oversampling TEchnique (SMOTE) is known as the benchmark method to solve class imbalance learning. Since SMOTE was proposed, many variants of it have emerged, which are classified into two types: pre-processing and post-processing. However, most of the pre-processing methods do not filter the noisy samples; at the same time, the post-processing methods do not give attention to the focus area data. In this paper, we present an oversampling method based on kmeans-SMOTE and Iterative Partition Filter (KSIPF), which overcomes the shortcomings of the above methods. Firstly, KSIPF uses  $k$ -means to cluster the data and selects the clusters to oversample, and then, IPF is used to remove the noise samples from the data. Then, KSIPF is compared with the SMOTE and its variants on 30 synthetic imbalanced data sets and 20 real-world imbalanced data sets, and the balanced data sets are used to train SVM and AdaBoost classifiers to determine whether it is effective. Finally, the experiment results demonstrate that KSIPF performs better than the comparisons, including area under the curve, F1-measure, and the statistical test.

**Keywords** Imbalanced classification · Noise filter · Iterative partition filtering · SMOTE

---

✉ Zhiping Wang  
wzp@dlmu.edu.cn

✉ Xiaoxi Wang  
289514152@qq.com

<sup>1</sup> School of Science, Dalian Maritime University, Dalian 116026, Liaoning, China

<sup>2</sup> Department of Clinical Laboratory Medicine, The First Affiliated Hospital of Dalian Medical University, Dalian 116011, Liaoning, China

## 1 Introduction

The number of samples in different classes is often unequal and dissymmetric in data sets, especially in real-world data sets. This is also an important reason that affects the reliability of classifiers. Indeed, unequal distribution of data causes some problems such as class overlapping, small sample sizes, or small disjuncts that damage the classifier learning [1–3]. The imbalance between classes will seriously reduce the accuracy of classification because they usually ignore the features of minority classes. Therefore, to improve the accuracy of the classifier, many scholars [4–7] have studied the imbalanced data. The existing methods are roughly divided into the following strategies: the strategies at the data level, the strategies at the algorithm level [8–11], and the cost-sensitive strategy that combines the above two methods [12–14].

Due to the generality of the data-level methods, they are independent of the classifiers. Moreover, data-level methods can pre-process all data sets in advance to train different classifiers. So, the computation time required to prepare the data is only once. And the data-level methods have the advantages of being easy to operate and unaffected by the used classification model. For these reasons, the data-level methods are the most prevalent methods, and the proposed method, KSIPF, in this paper belongs to the data-level methods.

The most popular data-level method is Synthetic Minority Oversampling TEchnique (SMOTE) [15]. However, SMOTE is a randomized algorithm that does not pay attention to the distribution and structure of the data. To improve the performance of this algorithm, many SMOTE variants methods have been introduced, such as SMOTE-ENN [16], kmeans-SMOTE [17], IW-SMOTE [18], SW-SMOTE [19], TDMO [20], SMOTE-kTLNN [21], and so on. Nevertheless, these methods either only focus on specific areas without denoising or filter out too many samples without considering the distribution of data.

To achieve superior performance, an advanced oversampling method based on  $k$ -means clustering and Iterative Partition Filter (KSIPF) has been developed. This method integrates both data distribution information and noise filtering capabilities. Specifically, the process begins with clustering the data into  $k$  distinct clusters. Subsequently, clusters that contain a dominant number of minority class samples are identified, and new minority class samples are synthesized within these clusters. Finally, the oversampled data are refined through the Iterative Partition Filter (IPF) [22]. As a result, KSIPF demonstrates highly encouraging outcomes on both 30 synthetic data sets and 20 real-world data sets, thereby establishing its significant superiority.

The main contributions of this paper are summarized as follows:

- We put forward a new method KSIPF that combines pre-process and post-process to achieve robust results, which solves the problem of generating a large number of noise samples.
- Compared with the Edited Nearest Neighbor (ENN) filtering method, KSIPF generates better data sets. This is because KSIPF incorporates iterative pro-

cesses to effectively remove noisy samples. Additionally, the hyperparameters in KSIPF are set in a highly reasonable manner.

- A large number of experiments are carried out to prove the effectiveness of KSIPF. It is compared with nine oversampling methods on different benchmark data sets. KSIPF has an average increase of 1–5% on AUC, F1-measure, and has passed the nonparametric hypothesis test.

The rest of this paper is organized as follows. In Sect. 2, we will review the literature of imbalanced data classification. Section 3 introduces the basic concepts and algorithms. In Sect. 4, the experimental framework is defined, and the results obtained from experiments are analyzed in Sect. 5. Finally, in Sect. 6, we conclude the paper.

## 2 Related work

### 2.1 The problem of imbalanced data sets

The main difficulty for imbalanced data sets is that the standard classifier ignores the importance of minority samples in the process of classification. Consequently, the minority samples are more likely to be misclassified than majority. This type of data may be categorized depending on its imbalance ratio (IR), which is defined as the relation between the majority class and minority class samples, by Eq. (1):

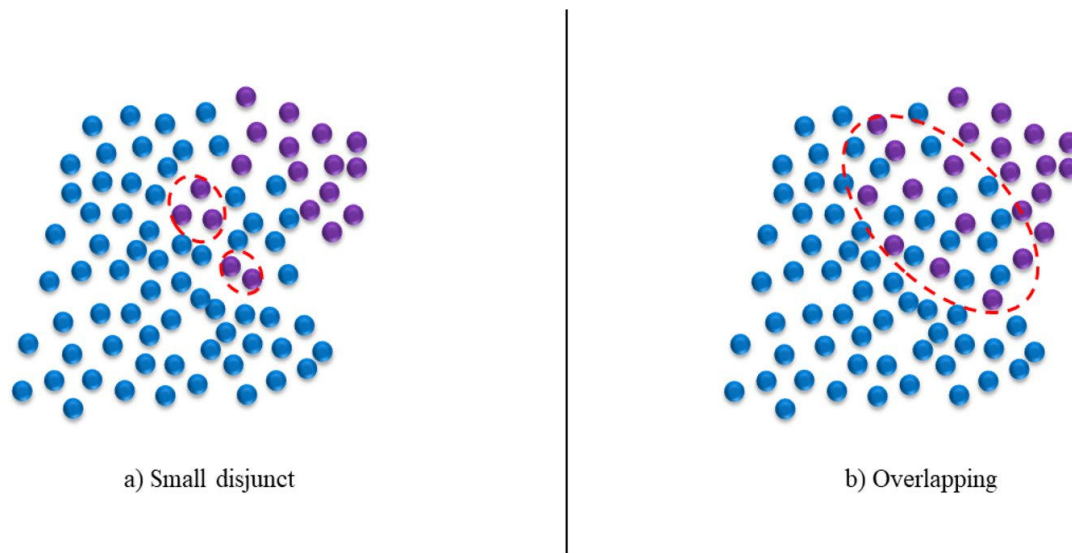
$$IR = \frac{N^-}{N^+}, \quad (1)$$

where  $N^-$  and  $N^+$  are the number of samples belonging to the majority and minority classes, respectively. When  $IR > 1$ , the data set is called an imbalanced data set.

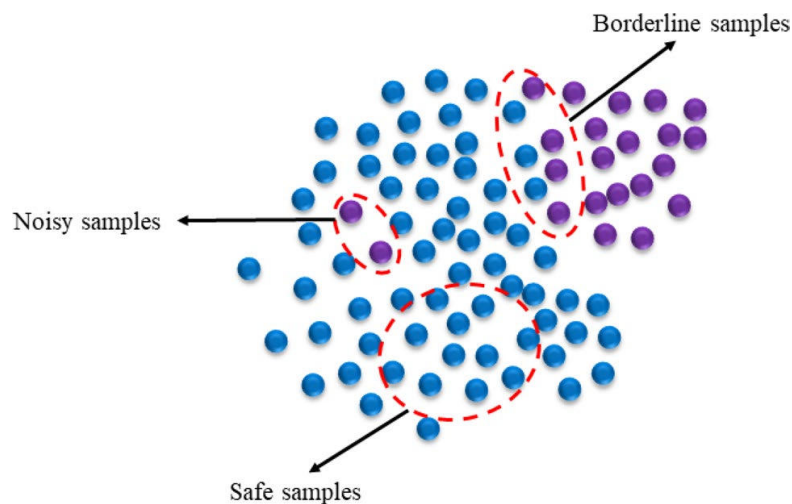
The imbalance ratio is not the only difficulty faced by the classifier, but there are also other problems that affect the classification performance: presence of small disjuncts [23, 24] and overlapping between classes [25] are shown in Fig. 1a, b, respectively. Figure 1a shows the small disjuncts, where the minority class samples are divided into many subsets, while the number of samples in each subset is small and surrounded by the majority class samples. Figure 1b shows the overlapping, where the samples are located at the class boundaries. Since samples from different classes may have similar characteristics, the samples are surrounded by samples that do not belong to their own classes, i.e., the samples overlap between classes. Usually, misclassification often occurs in the class boundary region [26], where the class overlap presents.

In this paper, in order to perform noise filtering, the definition of sample types are given to facilitate noise filtering. To better illustrate this process, and classify the samples in the data set, every sample belongs to one of the following three types [26] (Fig. 2):

1. Safe samples: samples surrounded by samples belonging to the same class.



**Fig. 1** Examples of the imbalance between classes



**Fig. 2** Three types of samples in the data set with two classes

2. Borderline samples: samples that are in the boundary region between two classes. The number of samples of both classes in their neighboring samples is approximately the same.
3. Noisy samples: samples surrounded by samples belonging to different classes.

## 2.2 SMOTE and its variants

Chawla et al. [15] have introduced the famous oversampling method SMOTE. In this method, the number of majority and minority samples is balanced by generating new minority samples. As given in Eq. (2), where  $x_i$  is a sample from the minority class, and  $x_j$  is one of the nearest neighbor samples,  $x_{new}$  is the newly generated

sample, and  $\text{rand}(0, 1)$  represents a random number between 0 and 1, the new samples are located between the lines of minority samples and their  $k$ -nearest neighbors.

$$x_{\text{new}} = x_i + \text{rand}(0,1) \times (x_j - x_i), j = 1, 2, \dots, M. \quad (2)$$

### 2.2.1 Pre-processing

These methods emphasize the samples in a specific area to make the samples more representative before sampling. Han et al. [27] designed Borderline-SMOTE to enhance the boundary area by investigating the domain information of minority classes. However, Borderline-SMOTE only selects minority class samples that have more heterogeneous samples nearby. In response to the defects of Borderline-SMOTE, Liu [28] proposed Importance-SMOTE to obtain better results. A novel Adaptive Synthetic Sampling algorithm (ADASYN) [29] is proposed, which adaptively determines the number of samples to generate. Inspired by ADASYN, many techniques incorporate similar mechanisms to be introduced [30, 31]. Safe-Level SMOTE [32] supports safer inner regions by measuring the safe-level ratio of minority samples and then synthesizing more instances in regions with higher safety levels. Yan et al. [33] combined the three-way decision model with the constructive covering algorithm (CCA) to detect boundary samples. Nevertheless, region-only methods not only ignore important information, but also lead to oversampling and overfitting. Chen et al. [34] proposed a robust method called RSMOTE. Weighting-based methods usually introduce additional hyperparameters, as described in [29, 32, 35], or focus only on allocating the number of composite samples [34]. Jia et al. [20] designed an oversampling method (TDMO) for locally distributed information detection based on extreme gradient boosting (XGBoost) and dynamic multidimensional oversampling. Meantime, it is also easy to generate noise in these pre-processing methods.

### 2.2.2 Post-processing

These extended methods refer to filtering noise samples after synthesizing new samples to improve the performance. Batista et al. [16] proposed two methods, SMOTE-Tomek Links and SMOTE-ENN, which first introduced the data cleaning methods into the filtering synthetic samples. Ramentol and Caballero et al. [36] combined rough set theory with approximate editing techniques in subsets, then proposed SMOTE-RSB. Sáez et al. [37] introduced a noise filter based on iterative ensemble, called iterative-partitioning filter (IPF). Xia et al. [38] proposed an efficient learning framework based on noise filtering (CRF-NFL), which uses CRF to divide the sample space. Chen et al. [39] put in MCRF and MRD, which combine SMOTE for multi-classification. Zhang et al. [40] proposed SMOTE-RkNN, which uses reverse  $k$ -nearest neighbors to identify the distribution density of samples. According to the fact that the density of noise in other classes is higher than the class it belongs to, noise samples are found and filtered, and good performance is obtained. SMOTE-kTLNN [21] innovated the use of a more efficient classifier, a two-layer nearest

neighbor classifier, which utilizes information from two layers of nearest neighbors to more accurately identify noisy samples, thereby improving the quality of synthetic samples.

### 3 Proposed method

Aiming to obtain data for the classifiers, an accurate oversampling method KSIPF is established. The proposed method can be divided into three stages: the pre-processing stage, the oversampling stage, and the filtering stage.

#### 3.1 The pre-processing stage

In this stage,  $k$ -means is used for clustering. The  $k$ -means cluster simply means that similar data is divided into one cluster. The similarity measure of  $k$ -means uses a distance measure, generally Euclidean distance.

The  $k$ -means clustering method initializes  $k$  clustering centers and divides each cluster by calculating the distance from the samples to the centroid. The distance between the samples and the center of the cluster class to which it belongs Eq. (3):

$$\underset{c}{\operatorname{argmin}} J(c) = \sum_{k=1}^K \sum_{x^{(i)} \in \mathbb{C}_k} \|x^{(i)} - \mu^{(k)}\|_2^2, \quad (3)$$

where  $c$  is the class label of the sample,  $\mathbb{C}_k$  is the set of samples,  $x^{(i)}$  denotes the  $i$ th sample, and  $\mu^{(k)}$  denotes the  $i$ th cluster center. Update the value of the cluster center as the average of the samples of the same class in Eq. (4):

$$\mu^{(k)} = \frac{\sum_{k=1}^m I\{c^{(k)} = j\} x^{(k)}}{\sum_{k=1}^m I\{c^{(k)} = j\}}, \quad (4)$$

where if  $c^{(k)} = j$ ,  $I\{c^{(k)} = j\} = 1$ , if  $c^{(k)} \neq j$ ,  $I\{c^{(k)} = j\} = 0$ .

Its algorithm steps are as follows:

1. Randomly select  $k$  samples as the initial cluster centers;
2. Calculate the distance of each sample from the  $k$  cluster centers and categorize the samples into the classes to which the closest cluster centers belong;
3. For the categorized samples, recalculate the new cluster centers;
4. Repeat the steps in (2)(3) until the stopping condition is reached.

As we all know, a key issue in the  $k$ -means clustering algorithm is the determination of the  $k$  value, that is, the number of clusters. Since oversampling technology is used on different data sets, a fixed  $k$  value cannot be applied to all data sets. Therefore,  $k$  is set to  $\sqrt{N_{\min}}$  in this paper, where  $N_{\min}$  is the number of minority



class samples in the data set. According to the above  $k$  settings, it is not difficult to see that different data sets correspond to different  $k$  values, and they will change according to the size of the data set, which can better reflect the characteristics of different data sets.

### 3.2 The oversampling stage

After the pre-processing stage, the clusters to be oversampled are selected. This phase is mainly inspired by selecting method in the literature by Douzas et al. [17]. The motivation for this step is to oversample only the minority-dominated clusters since it is safer to apply SMOTE in minority regions. Therefore, more generated samples are assigned to the sparse minority clusters than to the dense clusters [17]. The clusters selected for oversampling are mainly determined by comparing the imbalance ratio and the threshold value for each cluster. By default, the threshold for the imbalance ratio is set to 1. Cluster imbalance rate is defined as Eq. (5):

$$\text{imbalance ratio} = \frac{\text{majority count}(C_i) + 1}{\text{minority count}(C_i) + 1}. \quad (5)$$

After selecting the clusters to be oversampled, it is determined how many samples need to be generated for each class. The formula for calculating the weights is (6):

$$\text{sampling weight}(f) = \frac{\text{sparsity factors}(f)}{\text{sparsity sum}}. \quad (6)$$

In the above equation, the sparsity factor denotes the sparsity metric factor, which is the reciprocal of the density metric (7):

$$\text{density factors} = \frac{\text{minority count}(f)}{\text{average minority distance}(f)^m}. \quad (7)$$

The selected classes are oversampled to achieve a target ratio of minority to majority instances. The number of samples generated is calculated using the sampling weights (8):

$$\text{number of samples} = \|n \times \text{sampling weight}(f)\|, \quad (8)$$

where  $n$  is the difference between the number of majority samples and the number of minority samples. The above steps complete oversampling by interpolating in clusters with higher IR values, which can avoid generating noise samples between different clusters. However, there is also a problem that the existing noise samples are not filtered, which will also cause damage to the classification model. To solve this problem, this paper introduces a stage of filtering noise samples after oversampling.

### 3.3 The filtering stage

Borderline samples and noisy samples often make the standard classifier erroneous [41]. Therefore, detecting and removing the worst of the noisy samples will make the final classifier more efficient. So, we give some filters: Edited Nearest Neighbor (ENN) [42], Ensemble Filter [43], the Cross-Validated Committees Filter (CVCF) [44], the iterative-partitioning filter (IPF) [22]. The IPF achieved optimal results, so it is used in this paper. The details of IPF are as follows: The method starts with the set of noisy sample instances  $A = \emptyset$ .

1. Partition the current training data set into  $n$  subsets of equal size.
2. Use C4.5 to build a classifier on each subset, then use them to evaluate the current training data set  $E$ .
3. Add the samples identified as noisy in  $E$  to  $A$  using a voting scheme to remove the noisy samples:  $E = E \setminus A$ .

Repeat the steps in (2)(3) until the specified number of iterations  $T = 3$  is reached or the number of noisy samples identified in each iteration is less than  $p = 1\%$  of the data set size [37]. There are two voting schemes used in the iterative process: consensus (only when all classification results are the same will they not be identified as noise samples) and majority (when most of the classification results are different from the original labels, they are identified as noise samples).

The details of the pseudo-code of the proposed KSIPF method are shown in Algorithm 1.



**Algorithm 1** KSIPF algorithm.

---

**Input:**  $X$  (matrix of observations),  $y$  (target),  $n$  (number of samples to be generated),  
 $k$  (number of clusters to be found by k-means),  $irt$  (imbalance ratio threshold),  
 $knn$  (number of nearest neighbors considered by SMOTE),  $m$  (exponent of density),  
 $nn$  (number of subsets)

1 **Begin**

2 //The pre-processing stage.

3  $clusters \leftarrow k\text{-means}(X)$

4  $filtered\ clusters \leftarrow \emptyset$

5 **for**  $c \in clusters$  **do**

6      $imbalance\ ratio \leftarrow \frac{majority\ count(c)+1}{minority\ count(c)+1}$

7     **if**  $imbalance\ ratio < irt$  **then**

8          $filtered\ clusters \leftarrow filtered\ clusters \cup \{c\}$

9     **end**

10 **end**

11 //The over-sampling stage.

12 **for**  $f \in filtered\ clusters$  **do**

13      $average\ minority\ distance(f) \leftarrow mean(Euclidean\ distances(f))$

14      $density\ factor(f) \leftarrow \frac{minority\ count(f)}{average\ minority\ distance(f)^m}$

15      $sparsity\ factor(f) \leftarrow \frac{1}{density\ factor(f)}$

16 **end**

17  $sparsity\ sum \leftarrow \sum_{f \in filtered\ clusters} sparsity\ factor(f)$

18  $sampling\ weight(f) \leftarrow \frac{sparsity\ factor(f)}{sparsity\ sum}$

19  $generated\ samples \leftarrow \emptyset$

20 **for**  $f \in filtered\ clusters$  **do**

21      $number\ of\ samples \leftarrow \lceil n \times sampling\ weight(f) \rceil$

22      $generated\ samples \leftarrow generated\ samples \cup \{SMOTE(f)\}$

23 **end**

24 //The filtering stage.

25  $subsets \leftarrow \frac{generated\ samples}{nn}, C4.5.fit(subsets), remove \leftarrow \emptyset$

26 **for**  $x \in generated\ samples$  **do**

27      $y_{pre} \leftarrow C4.5(x)$

28     **if**  $vote(y_{pre} \neq y)$  **then**

29          $remove \leftarrow remove \cup x$

30     **end**

31 **end**

32  $generated\ samples \leftarrow generated\ samples \setminus remove$

33 **Results: generated samples**

---

### 3.4 Complexity analysis

The  $k$ -means algorithm usually uses an iterative method to update the cluster centers and the attribution of samples. Each iteration needs to calculate the distance from each sample to all cluster centers. If there are  $N$  samples and  $K$  cluster centers, the time complexity of each iteration is  $O(NK)$ . Generally, it takes  $I$  iterations to converge, so the time complexity of the  $k$ -means algorithm is  $O(NKI)$ . For each minority class sample, its  $k$ -nearest neighbors must be calculated. If the number of samples is  $N_{min}$  (the number of minority class samples) and the feature dimension is  $d$ , the time complexity can be reduced to  $O(N_{min} \log N_{min})$ . When generating new samples, the time complexity is  $O(d)$ . Assuming that  $p$  new samples are to be generated, the time complexity of generating new samples is  $O(pd)$ . Therefore, the total time complexity of the SMOTE part is  $O(N_{min} \log N_{min})$ . The total number of samples is  $(N + p)$ , the number of iterations is  $T$ , and each iteration divides the training set into  $n$  subsets. The time complexity of building a decision tree is  $O((N + p) \log(N + p))$ , and the time complexity of classification is  $O(N + p)$ . Then, the time complexity of the IPF part is  $O(Tn(N + p) \log(N + p) + (N + p))$ . The overall complexity is  $O(NKI + N_{min} \log N_{min} + Tn(N + p) \log(N + p) + (N + p))$ .

**Table 1** Specifications of the real-word data sets

| Number | Data sets                 | Samples | Dimension | Majority | Minority | IR    |
|--------|---------------------------|---------|-----------|----------|----------|-------|
| D1     | pima                      | 768     | 8         | 500      | 268      | 1.87  |
| D2     | yeast1                    | 1484    | 10        | 1055     | 429      | 2.46  |
| D3     | haberman                  | 306     | 3         | 225      | 81       | 2.78  |
| D4     | vehicle3                  | 846     | 18        | 634      | 212      | 2.99  |
| D5     | ecoli1                    | 336     | 7         | 259      | 77       | 3.36  |
| D6     | glass6                    | 214     | 9         | 185      | 29       | 6.38  |
| D7     | ecoli_0_6_7_vs_5          | 220     | 6         | 200      | 20       | 10    |
| D8     | ecoli_0_1_4_6_vs_5        | 280     | 6         | 260      | 20       | 13    |
| D9     | yeast_1_vs_7              | 459     | 7         | 427      | 30       | 14.3  |
| D10    | page_blocks_1_3_vs_4      | 472     | 10        | 444      | 28       | 15.86 |
| D11    | abalone9_18               | 731     | 8         | 689      | 42       | 16.4  |
| D12    | hypothyroid               | 3163    | 25        | 3012     | 151      | 19.95 |
| D13    | car_vgood                 | 1729    | 6         | 1663     | 65       | 25.58 |
| D14    | abalone_3_vs_11           | 502     | 8         | 487      | 15       | 32.47 |
| D15    | yeast5                    | 1484    | 10        | 1440     | 44       | 32.73 |
| D16    | abalone_21_vs_8           | 581     | 8         | 567      | 14       | 40.5  |
| D17    | winequality_red_8_vs_6_7  | 855     | 11        | 837      | 18       | 46.5  |
| D18    | abalone_19_vs_10_11_12_13 | 1622    | 8         | 1590     | 32       | 49.69 |
| D19    | abalone_20_vs_8_9_10      | 1916    | 8         | 1890     | 26       | 72.69 |
| D20    | poker_8_vs_6              | 1477    | 25        | 1460     | 17       | 85.88 |

**Table 2** Confusion matrix

| Actual   | Predicted           |                     |
|----------|---------------------|---------------------|
|          | Positive            | Negative            |
| Positive | TP (True Positive)  | FN (False Negative) |
| Negative | FP (False Positive) | TN (Ture Negative)  |

4 Experimental framework

4.1 Synthetic imbalanced data sets

The performance of classification algorithms is significantly affected by the shape of the boundaries between classes. Therefore, we use synthetic data sets proposed by [26]. These data sets can be divided into three categories according to their shape: Sub-cluster, Clover, and Paw.

- Sub-cluster: The examples from the minority class are located inside rectangles following related works on small disjuncts.
- Clover: Minority class samples are like a flower with oval petals, representing more complex nonlinear settings.
- Paw: There is a great overlap between the majority samples and the minority samples.

The details of these data sets are described below, and more details can be found in [45]:

Attributes and classes: These data fall into two classes, both of which have two attributes. The numbers of samples and IR (imbalance ratio, that is, the ratio of the number of majority and minority classes): These data sets are divided into two types, 600 samples with  $IR = 5$  and 800 samples with  $IR = 7$ . Disturbance ratio (DR): The influence of the boundary samples on the classification algorithm is very significant, and in the definition of synthetic data sets, different levels of disturbance of boundary data have been utilized, include 0%, 30%, 50%, 60%, and 70%. These values are adjusted from [45].

4.2 Real-world imbalanced data sets

We use 20 real-world data sets for comparison from UCI and KEEL data set repositories. The detailed specifications of these sets are given in Table 1.

4.3 Set-up

In this paper, the methods below are used to compared with KSIPF: the SMOTE itself [15], B1-SMOTE [27], kmeans-SMOTE [17], B2-SMOTE [27],

SMOTE-IPF [37], SMOTE-ENN [16], KNNOR [46], SyMProD [47], and SMO-TECDNN [48].

The SVM and AdaBoost classifiers are used as the final classifier to evaluate the methods. Since the correct classification of positive samples is critical in imbalanced data classification algorithms, accuracy is not an appropriate performance measure [49]. Therefore, in this paper, we used the value of the area under the curve (AUC) [50] as the evaluation index. AUC is the criterion for judging the merit of the binary prediction model. In addition, we also use F1-measure to measure the ability of oversampling algorithm. Its expression is as follows, and it is defined according to the confusion matrix shown in Table 2.

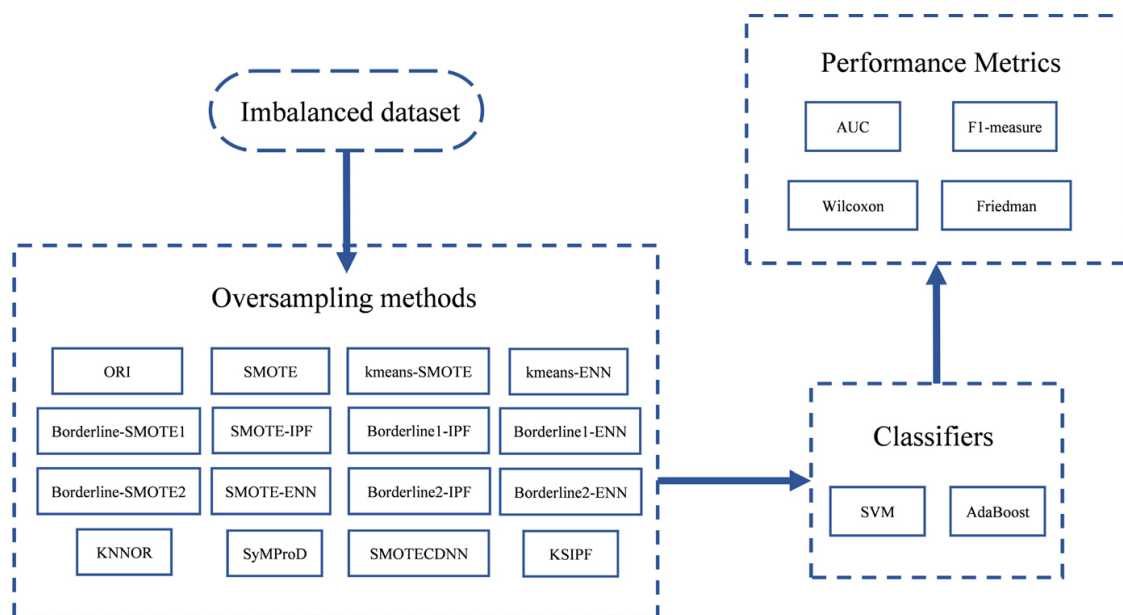
$$Precision = \frac{TP}{TP + FP}, \quad (9)$$

$$Recall/Sensitivity = \frac{TP}{TP + FN}, \quad (10)$$

$$F1 - measure = \frac{2 \times Recall \times Precision}{Recall + Precision}, \quad (11)$$

$$AUC = \frac{\sum_{i \in positive} rank(i) - \frac{M(M+1)}{2}}{M \times N}. \quad (12)$$

Among them, Precision represents the proportion of the predicted positive minority samples that are actually positive minority samples. The Recall is also called



**Fig. 3** Illustration of the experimental process. Firstly, the imbalanced data are oversampled, and then, the classifiers are trained on the train data set, and the trained classifiers are used to predict the test set. Get the results in AUC and F1-measure. Finally, statistical hypothesis testing is performed on the obtained results

**Table 3** The AUC results obtained by the SVM classifier on the synthetic data sets. (e.g., ORI: without oversampling, kmeans: kmeans-SMOTE; B1: Borderline-SMOTE1, ENN: SMOTE-ENN, and IPF: SMOTE-IPF)

| Data sets                     | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        | Combining both pre and post |        |        |            |        |        |
|-------------------------------|--------|--------|----------------|--------|--------|-----------------|--------|-----------------------------|--------|--------|------------|--------|--------|
|                               |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPF                       | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
|                               |        |        |                |        |        |                 |        |                             |        |        |            |        |        |
| subcl <sub>IR=5, DR=0</sub>   | 0.9230 | 0.9491 | 0.9671         | 0.9496 | 0.9285 | 0.9711          | 0.9509 | 0.9543                      | 0.9844 | 0.9801 | 0.9571     | 0.9705 | 0.9653 |
| subcl <sub>IR=5, DR=30</sub>  | 0.8738 | 0.9168 | 0.9416         | 0.9119 | 0.8815 | 0.9810          | 0.9182 | 0.9392                      | 0.9807 | 0.9875 | 0.9363     | 0.9892 | 0.9758 |
| subcl <sub>IR=5, DR=50</sub>  | 0.8475 | 0.8980 | 0.9368         | 0.9044 | 0.8773 | 0.9778          | 0.8958 | 0.9419                      | 0.9826 | 0.9714 | 0.9411     | 0.9786 | 0.9660 |
| subcl <sub>IR=5, DR=60</sub>  | 0.8476 | 0.8925 | 0.9310         | 0.8888 | 0.8587 | 0.9744          | 0.8878 | 0.9265                      | 0.9734 | 0.9654 | 0.9436     | 0.9725 | 0.9563 |
| subcl <sub>IR=5, DR=70</sub>  | 0.8172 | 0.8798 | 0.9305         | 0.8902 | 0.8694 | 0.9778          | 0.8721 | 0.9535                      | 0.9837 | 0.9653 | 0.9341     | 0.9799 | 0.9716 |
| subcl <sub>IR=7, DR=0</sub>   | 0.9332 | 0.9470 | 0.9557         | 0.9485 | 0.9267 | 0.9632          | 0.9473 | 0.9577                      | 0.9661 | 0.9722 | 0.9498     | 0.9608 | 0.9561 |
| subcl <sub>IR=7, DR=30</sub>  | 0.8242 | 0.9199 | 0.9477         | 0.9257 | 0.8960 | 0.9781          | 0.9251 | 0.9513                      | 0.9858 | 0.9668 | 0.9333     | 0.9869 | 0.9672 |
| subcl <sub>IR=7, DR=50</sub>  | 0.7592 | 0.8977 | 0.9291         | 0.9106 | 0.8748 | 0.9729          | 0.8981 | 0.9416                      | 0.9878 | 0.9746 | 0.9262     | 0.9699 | 0.9560 |
| subcl <sub>IR=7, DR=60</sub>  | 0.6976 | 0.8911 | 0.9275         | 0.8925 | 0.8586 | 0.9585          | 0.8922 | 0.9196                      | 0.9630 | 0.9479 | 0.9158     | 0.9534 | 0.9328 |
| subcl <sub>IR=7, DR=70</sub>  | 0.6822 | 0.8699 | 0.9148         | 0.9005 | 0.8574 | 0.9616          | 0.8725 | 0.8811                      | 0.9673 | 0.9422 | 0.9121     | 0.9704 | 0.9454 |
| clover <sub>IR=5, DR=0</sub>  | 0.848  | 0.9311 | 0.9592         | 0.9403 | 0.9299 | 0.9496          | 0.9372 | 0.9865                      | 0.9706 | 0.9611 | 0.9685     | 0.9614 | 0.9569 |
| clover <sub>IR=5, DR=30</sub> | 0.8339 | 0.9183 | 0.9456         | 0.9203 | 0.8923 | 0.9607          | 0.9169 | 0.9976                      | 0.9592 | 0.9555 | 0.9871     | 0.9665 | 0.9520 |
| clover <sub>IR=5, DR=50</sub> | 0.8301 | 0.8902 | 0.9415         | 0.8945 | 0.8823 | 0.9661          | 0.8942 | 0.9976                      | 0.9644 | 0.9692 | 0.9896     | 0.9659 | 0.9723 |
| clover <sub>IR=5, DR=60</sub> | 0.8438 | 0.8947 | 0.9420         | 0.8885 | 0.8597 | 0.9752          | 0.8914 | 0.9971                      | 0.9622 | 0.9563 | 0.9811     | 0.9722 | 0.9617 |
| clover <sub>IR=5, DR=70</sub> | 0.8113 | 0.8831 | 0.9394         | 0.8805 | 0.8619 | 0.9640          | 0.8800 | 0.9982                      | 0.9726 | 0.9496 | 0.9882     | 0.9635 | 0.9627 |
| clover <sub>IR=7, DR=0</sub>  | 0.9079 | 0.9422 | 0.9646         | 0.9489 | 0.9413 | 0.9587          | 0.9486 | 0.9896                      | 0.9691 | 0.9671 | 0.9650     | 0.9662 | 0.9660 |
| clover <sub>IR=7, DR=30</sub> | 0.8838 | 0.9230 | 0.9577         | 0.9261 | 0.9045 | 0.9588          | 0.9227 | 0.9987                      | 0.9704 | 0.9593 | 0.9834     | 0.9645 | 0.9516 |
| clover <sub>IR=7, DR=50</sub> | 0.8765 | 0.9020 | 0.9550         | 0.9035 | 0.8812 | 0.9632          | 0.9015 | 0.9976                      | 0.9627 | 0.9596 | 0.9884     | 0.9605 | 0.9559 |
| clover <sub>IR=7, DR=60</sub> | 0.8666 | 0.8970 | 0.9208         | 0.9101 | 0.8804 | 0.9676          | 0.9013 | 0.9965                      | 0.9627 | 0.9538 | 0.9862     | 0.9616 | 0.9560 |
| clover <sub>IR=7, DR=70</sub> | 0.8333 | 0.8906 | 0.9454         | 0.8949 | 0.8710 | 0.9590          | 0.8818 | 0.9774                      | 0.9581 | 0.9503 | 0.8675     | 0.9588 | 0.9440 |
| paw <sub>IR=5, DR=0</sub>     | 0.9585 | 0.9906 | 0.9922         | 0.9882 | 0.9818 | 0.9971          | 0.9890 | 0.9922                      | 0.9949 | 0.9943 | 0.9966     | 0.9941 | 0.9931 |

**Table 3** (continued)

| Data sets                  | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        | Combining both pre and post |        |        |            |        |        |
|----------------------------|--------|--------|----------------|--------|--------|-----------------|--------|-----------------------------|--------|--------|------------|--------|--------|
|                            |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPIF                      | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
|                            |        |        |                |        |        |                 |        |                             |        |        |            |        |        |
| paw <sub>IR=5, DR=30</sub> | 0.9322 | 0.9664 | 0.9778         | 0.9540 | 0.9389 | 0.9881          | 0.9657 | 0.9977                      | 0.9853 | 0.9769 | 0.9888     | 0.9812 | 0.9743 |
| paw <sub>IR=5, DR=50</sub> | 0.9211 | 0.9531 | 0.9680         | 0.9394 | 0.9218 | 0.9915          | 0.9510 | 0.9998                      | 0.9905 | 0.9914 | 0.9969     | 0.9893 | 0.9857 |
| paw <sub>IR=5, DR=60</sub> | 0.9060 | 0.9408 | 0.9501         | 0.9241 | 0.9002 | 0.9850          | 0.9393 | 0.9987                      | 0.9832 | 0.9747 | 0.9899     | 0.9723 | 0.9703 |
| paw <sub>IR=5, DR=70</sub> | 0.9054 | 0.9299 | 0.9597         | 0.9166 | 0.9038 | 0.9891          | 0.9299 | 0.9990                      | 0.9889 | 0.9945 | 0.9966     | 0.9866 | 0.9881 |
| paw <sub>IR=7, DR=0</sub>  | 0.9592 | 0.9938 | 0.9930         | 0.9868 | 0.9777 | 0.9987          | 0.9930 | 0.9985                      | 0.9944 | 0.9877 | 0.9972     | 0.9923 | 0.9882 |
| paw <sub>IR=7, DR=30</sub> | 0.9267 | 0.9655 | 0.9680         | 0.9615 | 0.9405 | 0.9902          | 0.9655 | 0.9992                      | 0.9892 | 0.9664 | 0.9896     | 0.9819 | 0.9708 |
| paw <sub>IR=7, DR=50</sub> | 0.9165 | 0.9462 | 0.9636         | 0.9350 | 0.9198 | 0.9910          | 0.9506 | 0.9995                      | 0.9861 | 0.9718 | 0.9939     | 0.9808 | 0.9747 |
| paw <sub>IR=7, DR=60</sub> | 0.8809 | 0.9246 | 0.9657         | 0.9253 | 0.9011 | 0.9807          | 0.9270 | 0.9995                      | 0.9757 | 0.9751 | 0.9873     | 0.9709 | 0.9626 |
| paw <sub>IR=7, DR=70</sub> | 0.8836 | 0.9215 | 0.9653         | 0.9121 | 0.9026 | 0.9857          | 0.9236 | 0.9997                      | 0.9854 | 0.9698 | 0.9926     | 0.9767 | 0.9813 |
| Average                    | 0.8644 | 0.9222 | 0.9519         | 0.9224 | 0.9007 | 0.9745          | 0.9223 | 0.9762                      | 0.9767 | 0.9686 | 0.9661     | 0.9733 | 0.9654 |



**Table 4** The F1-measure results obtained by the SVM classifier on the synthetic data sets. (e.g., ORI: without oversampling, kmeans: kmeans-SMOTE; B1: Borderline-SMOTE1, ENN: SMOTE-ENN, and IPF: SMOTE-IPF)

| Data sets                     | ORI    | SMOTE  | Pre-processing |        | Post-processing |        | Combining both pre and post |        |        |        |            |        |        |
|-------------------------------|--------|--------|----------------|--------|-----------------|--------|-----------------------------|--------|--------|--------|------------|--------|--------|
|                               |        |        | kmeans         | B1     | B2              | ENN    | IPF                         | KSIPF  | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
| subcl <sub>IR=5, DR=0</sub>   | 0.9085 | 0.8952 | 0.9069         | 0.8891 | 0.8547          | 0.9364 | 0.8975                      | 0.9228 | 0.9215 | 0.9054 | 0.9338     | 0.9270 | 0.9146 |
| subcl <sub>IR=5, DR=30</sub>  | 0.9091 | 0.8427 | 0.8633         | 0.8448 | 0.8191          | 0.9402 | 0.8444                      | 0.9617 | 0.9123 | 0.9182 | 0.9199     | 0.9315 | 0.9166 |
| subcl <sub>IR=5, DR=50</sub>  | 0.9091 | 0.8133 | 0.8665         | 0.8229 | 0.7917          | 0.9366 | 0.8168                      | 0.9587 | 0.9075 | 0.8879 | 0.9268     | 0.9196 | 0.8872 |
| subcl <sub>IR=5, DR=60</sub>  | 0.9091 | 0.8315 | 0.8568         | 0.8289 | 0.7951          | 0.9423 | 0.8212                      | 0.9646 | 0.9168 | 0.8973 | 0.9235     | 0.9262 | 0.8985 |
| subcl <sub>IR=5, DR=70</sub>  | 0.9091 | 0.7982 | 0.8535         | 0.8168 | 0.7852          | 0.9286 | 0.7883                      | 0.9672 | 0.9237 | 0.8960 | 0.9165     | 0.9137 | 0.8921 |
| subcl <sub>IR=7, DR=0</sub>   | 0.9333 | 0.9025 | 0.9007         | 0.8982 | 0.8684          | 0.9301 | 0.9047                      | 0.9492 | 0.9176 | 0.9054 | 0.9288     | 0.9281 | 0.9136 |
| subcl <sub>IR=7, DR=30</sub>  | 0.9333 | 0.8501 | 0.8780         | 0.8599 | 0.8307          | 0.9418 | 0.8553                      | 0.9685 | 0.9329 | 0.9106 | 0.9219     | 0.9317 | 0.9173 |
| subcl <sub>IR=7, DR=50</sub>  | 0.9333 | 0.8081 | 0.8600         | 0.8318 | 0.7816          | 0.9220 | 0.8173                      | 0.9745 | 0.9005 | 0.8874 | 0.9192     | 0.8999 | 0.8820 |
| subcl <sub>IR=7, DR=60</sub>  | 0.9333 | 0.8197 | 0.8541         | 0.8183 | 0.7952          | 0.9292 | 0.8325                      | 0.9799 | 0.9074 | 0.8887 | 0.9191     | 0.9133 | 0.8786 |
| subcl <sub>IR=7, DR=70</sub>  | 0.9333 | 0.7940 | 0.8490         | 0.8082 | 0.7827          | 0.9185 | 0.7989                      | 0.9849 | 0.8959 | 0.8767 | 0.9183     | 0.9127 | 0.8739 |
| clover <sub>IR=5, DR=0</sub>  | 0.9063 | 0.8357 | 0.8771         | 0.8446 | 0.8317          | 0.8683 | 0.8241                      | 0.9452 | 0.9008 | 0.8729 | 0.9104     | 0.8753 | 0.8780 |
| clover <sub>IR=5, DR=30</sub> | 0.9062 | 0.8335 | 0.8839         | 0.8197 | 0.7902          | 0.9030 | 0.8331                      | 0.9731 | 0.8672 | 0.8527 | 0.9286     | 0.9027 | 0.8840 |
| clover <sub>IR=5, DR=50</sub> | 0.9072 | 0.8159 | 0.8660         | 0.8214 | 0.7907          | 0.9213 | 0.8172                      | 0.9749 | 0.9114 | 0.9113 | 0.9306     | 0.9161 | 0.9145 |
| clover <sub>IR=5, DR=60</sub> | 0.9073 | 0.8151 | 0.8807         | 0.7883 | 0.7723          | 0.9450 | 0.8160                      | 0.9775 | 0.8912 | 0.8687 | 0.9342     | 0.9198 | 0.9063 |
| clover <sub>IR=5, DR=70</sub> | 0.9090 | 0.7980 | 0.8783         | 0.7987 | 0.7709          | 0.9033 | 0.7897                      | 0.9846 | 0.8935 | 0.8662 | 0.9359     | 0.8994 | 0.8919 |
| clover <sub>IR=7, DR=0</sub>  | 0.9333 | 0.8391 | 0.9142         | 0.8300 | 0.8141          | 0.8607 | 0.8354                      | 0.9538 | 0.8738 | 0.8495 | 0.9101     | 0.8569 | 0.8554 |
| clover <sub>IR=7, DR=30</sub> | 0.9333 | 0.8336 | 0.9109         | 0.8347 | 0.8126          | 0.8956 | 0.8367                      | 0.9886 | 0.9033 | 0.8866 | 0.9356     | 0.8871 | 0.8883 |
| clover <sub>IR=7, DR=50</sub> | 0.9333 | 0.8199 | 0.9081         | 0.8379 | 0.8179          | 0.9056 | 0.8256                      | 0.9847 | 0.9154 | 0.9001 | 0.9418     | 0.9108 | 0.9016 |
| clover <sub>IR=7, DR=60</sub> | 0.9333 | 0.8100 | 0.8518         | 0.8101 | 0.7939          | 0.9243 | 0.8160                      | 0.9807 | 0.9028 | 0.8821 | 0.9445     | 0.9098 | 0.8896 |
| clover <sub>IR=7, DR=70</sub> | 0.9333 | 0.8053 | 0.9002         | 0.8130 | 0.7933          | 0.8981 | 0.7997                      | 0.9821 | 0.8964 | 0.8724 | 0.9389     | 0.8978 | 0.8802 |
| paw <sub>IR=5, DR=0</sub>     | 0.9436 | 0.9429 | 0.9564         | 0.9364 | 0.9294          | 0.9586 | 0.9407                      | 0.9670 | 0.9627 | 0.9535 | 0.9540     | 0.9533 | 0.9515 |
| paw <sub>IR=5, DR=30</sub>    | 0.9269 | 0.9019 | 0.9273         | 0.8987 | 0.8687          | 0.9766 | 0.9023                      | 0.9828 | 0.9588 | 0.9352 | 0.9636     | 0.9599 | 0.9465 |

**Table 4** (continued)

| Data sets                  | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        | Combining both pre and post |        |        |            |        |        |
|----------------------------|--------|--------|----------------|--------|--------|-----------------|--------|-----------------------------|--------|--------|------------|--------|--------|
|                            |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPF                       | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
| paw <sub>IR=5, DR=50</sub> | 0.9223 | 0.8956 | 0.9171         | 0.8849 | 0.8628 | 0.9638          | 0.8950 | 0.9919                      | 0.9585 | 0.9577 | 0.9753     | 0.9551 | 0.9541 |
| paw <sub>IR=5, DR=60</sub> | 0.9207 | 0.8656 | 0.8888         | 0.8667 | 0.8484 | 0.9577          | 0.8694 | 0.9827                      | 0.9468 | 0.9441 | 0.9560     | 0.9541 | 0.9435 |
| paw <sub>IR=5, DR=70</sub> | 0.9203 | 0.8639 | 0.8953         | 0.8659 | 0.8419 | 0.9689          | 0.8684 | 0.9827                      | 0.9571 | 0.9442 | 0.9678     | 0.9576 | 0.9584 |
| paw <sub>IR=7, DR=0</sub>  | 0.9532 | 0.9490 | 0.9491         | 0.9445 | 0.9362 | 0.9635          | 0.9507 | 0.9837                      | 0.9546 | 0.9577 | 0.9713     | 0.9586 | 0.9584 |
| paw <sub>IR=7, DR=30</sub> | 0.9406 | 0.8972 | 0.9059         | 0.8976 | 0.8698 | 0.9579          | 0.8986 | 0.9867                      | 0.9450 | 0.9315 | 0.9569     | 0.9519 | 0.9374 |
| paw <sub>IR=7, DR=50</sub> | 0.9377 | 0.8777 | 0.9192         | 0.8894 | 0.8735 | 0.9619          | 0.8854 | 0.9881                      | 0.9496 | 0.9330 | 0.9574     | 0.9545 | 0.9439 |
| paw <sub>IR=7, DR=60</sub> | 0.9348 | 0.8520 | 0.9098         | 0.8650 | 0.8388 | 0.9508          | 0.8604 | 0.9903                      | 0.9295 | 0.9327 | 0.9490     | 0.9370 | 0.9231 |
| paw <sub>IR=7, DR=70</sub> | 0.9340 | 0.8562 | 0.9174         | 0.8553 | 0.8418 | 0.9651          | 0.8590 | 0.9916                      | 0.9517 | 0.9387 | 0.9564     | 0.9557 | 0.9447 |
| Average                    | 0.9249 | 0.8488 | 0.8915         | 0.8507 | 0.8268 | 0.9325          | 0.8500 | 0.9742                      | 0.9202 | 0.9055 | 0.9382     | 0.9239 | 0.9109 |

**Table 5** The AUC results obtained by the AdaBoost classifier on the synthetic data sets. (e.g., ORI: without oversampling, kmeans: kmeans-SMOTE; B1: Borderline-SMOTE1, ENN: SMOTE-ENN, and IPF: SMOTE-IPF)

| Data sets                     | ORI    | SMOTE  | Pre-processing |        | Post-processing |        | Combining both pre and post |        |        |        |            |        |        |
|-------------------------------|--------|--------|----------------|--------|-----------------|--------|-----------------------------|--------|--------|--------|------------|--------|--------|
|                               |        |        | kmeans         | B1     | B2              | ENN    | IPF                         | KSIPF  | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
|                               |        |        |                |        |                 |        |                             |        |        |        |            |        |        |
| subcl <sub>IR=5, DR=0</sub>   | 0.9835 | 0.9971 | 0.9963         | 0.9970 | 0.9824          | 0.9987 | 0.9974                      | 0.9759 | 0.9968 | 0.9978 | 0.9861     | 0.9978 | 0.9941 |
| subcl <sub>IR=5, DR=30</sub>  | 0.9113 | 0.9555 | 0.9633         | 0.9439 | 0.9142          | 0.9857 | 0.9446                      | 0.9413 | 0.9916 | 0.9935 | 0.9620     | 0.9895 | 0.9767 |
| subcl <sub>IR=5, DR=50</sub>  | 0.8784 | 0.9225 | 0.9459         | 0.9292 | 0.9014          | 0.9856 | 0.9212                      | 0.9493 | 0.9929 | 0.9866 | 0.9578     | 0.9839 | 0.9811 |
| subcl <sub>IR=5, DR=60</sub>  | 0.8730 | 0.9076 | 0.9388         | 0.9130 | 0.8956          | 0.9852 | 0.9053                      | 0.9401 | 0.9855 | 0.9801 | 0.9535     | 0.9815 | 0.9750 |
| subcl <sub>IR=5, DR=70</sub>  | 0.8495 | 0.8962 | 0.9367         | 0.9000 | 0.8876          | 0.9842 | 0.8969                      | 0.9259 | 0.9839 | 0.9707 | 0.9343     | 0.9797 | 0.9770 |
| subcl <sub>IR=7, DR=0</sub>   | 0.9834 | 0.9980 | 0.9954         | 0.9977 | 0.9855          | 0.9994 | 0.9981                      | 0.9761 | 0.9983 | 0.9953 | 0.9882     | 0.9984 | 0.9950 |
| subcl <sub>IR=7, DR=30</sub>  | 0.9176 | 0.9510 | 0.9723         | 0.9591 | 0.9365          | 0.9923 | 0.9629                      | 0.9735 | 0.9946 | 0.9827 | 0.9544     | 0.9869 | 0.9755 |
| subcl <sub>IR=7, DR=50</sub>  | 0.8888 | 0.9220 | 0.9424         | 0.9367 | 0.9108          | 0.9750 | 0.9212                      | 0.9647 | 0.9904 | 0.9889 | 0.9386     | 0.9728 | 0.9689 |
| subcl <sub>IR=7, DR=60</sub>  | 0.8630 | 0.9126 | 0.9377         | 0.9228 | 0.8975          | 0.9759 | 0.9059                      | 0.9505 | 0.9924 | 0.9790 | 0.9219     | 0.9667 | 0.9591 |
| subcl <sub>IR=7, DR=70</sub>  | 0.8510 | 0.8964 | 0.9293         | 0.9227 | 0.8989          | 0.9698 | 0.8967                      | 0.9220 | 0.9825 | 0.9813 | 0.9178     | 0.9782 | 0.9599 |
| clover <sub>IR=5, DR=0</sub>  | 0.8770 | 0.9313 | 0.9575         | 0.9456 | 0.9454          | 0.9519 | 0.9224                      | 0.9887 | 0.9755 | 0.9831 | 0.9694     | 0.9636 | 0.9744 |
| clover <sub>IR=5, DR=30</sub> | 0.8523 | 0.8979 | 0.9483         | 0.9017 | 0.8821          | 0.9571 | 0.9010                      | 0.9862 | 0.9783 | 0.9716 | 0.9745     | 0.9556 | 0.9522 |
| clover <sub>IR=5, DR=50</sub> | 0.8471 | 0.8869 | 0.9366         | 0.8888 | 0.8898          | 0.9711 | 0.8850                      | 0.9952 | 0.9730 | 0.9831 | 0.9825     | 0.9644 | 0.9671 |
| clover <sub>IR=5, DR=60</sub> | 0.8355 | 0.8827 | 0.9367         | 0.8644 | 0.8609          | 0.9766 | 0.8810                      | 0.9954 | 0.9632 | 0.9638 | 0.9802     | 0.9638 | 0.9495 |
| clover <sub>IR=5, DR=70</sub> | 0.8201 | 0.8727 | 0.9394         | 0.8652 | 0.8616          | 0.9633 | 0.8634                      | 0.9942 | 0.9750 | 0.9636 | 0.9813     | 0.9512 | 0.9469 |
| clover <sub>IR=7, DR=0</sub>  | 0.8698 | 0.9396 | 0.9740         | 0.9389 | 0.9345          | 0.9549 | 0.9317                      | 0.9925 | 0.9687 | 0.9714 | 0.9672     | 0.9567 | 0.9605 |
| clover <sub>IR=7, DR=30</sub> | 0.8548 | 0.8915 | 0.9561         | 0.9198 | 0.9041          | 0.9475 | 0.8943                      | 0.9962 | 0.9848 | 0.9646 | 0.9772     | 0.9511 | 0.9561 |
| clover <sub>IR=7, DR=50</sub> | 0.8420 | 0.8837 | 0.9536         | 0.9015 | 0.9011          | 0.9598 | 0.8895                      | 0.9969 | 0.9745 | 0.9708 | 0.9817     | 0.9581 | 0.9682 |
| clover <sub>IR=7, DR=60</sub> | 0.8352 | 0.8829 | 0.9300         | 0.8846 | 0.8759          | 0.9636 | 0.8879                      | 0.9983 | 0.9660 | 0.9737 | 0.9845     | 0.9535 | 0.9408 |
| clover <sub>IR=7, DR=70</sub> | 0.8132 | 0.8812 | 0.9479         | 0.8805 | 0.8807          | 0.9569 | 0.8753                      | 0.9602 | 0.9534 | 0.9623 | 0.8899     | 0.9468 | 0.9426 |
| paw <sub>IR=5, DR=0</sub>     | 0.9274 | 0.9524 | 0.9754         | 0.9612 | 0.9641          | 0.9612 | 0.9471                      | 0.9785 | 0.9820 | 0.9824 | 0.9537     | 0.9703 | 0.9770 |

Table 5 (continued)

| Data sets                  | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        | Combining both pre and post |        |        |            |        |        |
|----------------------------|--------|--------|----------------|--------|--------|-----------------|--------|-----------------------------|--------|--------|------------|--------|--------|
|                            |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPIF                      | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
| paw <sub>IR=5, DR=30</sub> | 0.8793 | 0.9081 | 0.9551         | 0.8858 | 0.8871 | 0.9664          | 0.9098 | 0.9950                      | 0.9589 | 0.9493 | 0.9792     | 0.9372 | 0.9380 |
| paw <sub>IR=5, DR=50</sub> | 0.8772 | 0.9078 | 0.9610         | 0.9053 | 0.9026 | 0.9695          | 0.9081 | 0.9958                      | 0.9741 | 0.9868 | 0.9887     | 0.9603 | 0.9658 |
| paw <sub>IR=5, DR=60</sub> | 0.8453 | 0.8912 | 0.9346         | 0.8916 | 0.8679 | 0.9622          | 0.8930 | 0.9924                      | 0.9697 | 0.9712 | 0.9720     | 0.9627 | 0.9511 |
| paw <sub>IR=5, DR=70</sub> | 0.8755 | 0.9002 | 0.9523         | 0.9044 | 0.9047 | 0.9812          | 0.9102 | 0.9961                      | 0.9828 | 0.9949 | 0.9865     | 0.9686 | 0.9745 |
| paw <sub>IR=7, DR=0</sub>  | 0.9232 | 0.9554 | 0.9620         | 0.9675 | 0.9679 | 0.9608          | 0.9536 | 0.9913                      | 0.9833 | 0.9843 | 0.9789     | 0.9738 | 0.9847 |
| paw <sub>IR=7, DR=30</sub> | 0.8863 | 0.9227 | 0.9504         | 0.9077 | 0.9261 | 0.9603          | 0.9231 | 0.9962                      | 0.9639 | 0.9646 | 0.9797     | 0.9440 | 0.9537 |
| paw <sub>IR=7, DR=50</sub> | 0.8834 | 0.9130 | 0.9539         | 0.9151 | 0.9259 | 0.9773          | 0.9198 | 0.9960                      | 0.9773 | 0.9830 | 0.9866     | 0.9631 | 0.9669 |
| paw <sub>IR=7, DR=60</sub> | 0.8461 | 0.8903 | 0.9526         | 0.9069 | 0.8960 | 0.9723          | 0.8881 | 0.9987                      | 0.9757 | 0.9735 | 0.9810     | 0.9705 | 0.9548 |
| paw <sub>IR=7, DR=70</sub> | 0.8620 | 0.8950 | 0.9604         | 0.9029 | 0.9036 | 0.9783          | 0.8944 | 0.9989                      | 0.9771 | 0.9807 | 0.9868     | 0.9676 | 0.9764 |
| Average                    | 0.8751 | 0.9148 | 0.9532         | 0.9187 | 0.9097 | 0.9715          | 0.9143 | 0.9787                      | 0.9789 | 0.9778 | 0.9665     | 0.9673 | 0.9655 |

**Table 6** The F1-measure results obtained by the AdaBoost classifier on the synthetic data sets. (e.g., ORI: without oversampling, kmeans: kmeans-SMOTE; B1: Borderline-SMOTE1, ENN: SMOTE-ENN, and IPF: SMOTE-IPF)

| Data sets                     | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        | Combining both pre and post |        |        |            |        |        |
|-------------------------------|--------|--------|----------------|--------|--------|-----------------|--------|-----------------------------|--------|--------|------------|--------|--------|
|                               |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPF                       | B1-IPF | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
|                               |        |        |                |        |        |                 |        |                             |        |        |            |        |        |
| subcl <sub>IR=5, DR=0</sub>   | 0.9878 | 0.9907 | 0.9812         | 0.9892 | 0.9475 | 0.9941          | 0.9909 | 0.9855                      | 0.9908 | 0.9938 | 0.9872     | 0.9866 | 0.9806 |
| subcl <sub>IR=5, DR=30</sub>  | 0.9293 | 0.8634 | 0.9068         | 0.8651 | 0.8511 | 0.9589          | 0.8516 | 0.9863                      | 0.9700 | 0.9833 | 0.9449     | 0.9593 | 0.9442 |
| subcl <sub>IR=5, DR=50</sub>  | 0.9043 | 0.8391 | 0.8872         | 0.8615 | 0.8386 | 0.9596          | 0.8371 | 0.9738                      | 0.9715 | 0.9566 | 0.9396     | 0.9409 | 0.9387 |
| subcl <sub>IR=5, DR=60</sub>  | 0.8955 | 0.8437 | 0.8793         | 0.8545 | 0.8364 | 0.9708          | 0.8438 | 0.9723                      | 0.9666 | 0.9580 | 0.9458     | 0.9503 | 0.9430 |
| subcl <sub>IR=5, DR=70</sub>  | 0.8963 | 0.8211 | 0.8706         | 0.8432 | 0.8274 | 0.9609          | 0.8238 | 0.9736                      | 0.9598 | 0.9394 | 0.9318     | 0.9519 | 0.9432 |
| subcl <sub>IR=7, DR=0</sub>   | 0.9918 | 0.9925 | 0.9817         | 0.9861 | 0.9503 | 0.9962          | 0.9929 | 0.9890                      | 0.9908 | 0.9869 | 0.9915     | 0.9907 | 0.9872 |
| subcl <sub>IR=7, DR=30</sub>  | 0.9501 | 0.8520 | 0.9135         | 0.8769 | 0.8704 | 0.9701          | 0.8802 | 0.9872                      | 0.9735 | 0.9515 | 0.9556     | 0.9533 | 0.9381 |
| subcl <sub>IR=7, DR=50</sub>  | 0.9338 | 0.8347 | 0.8742         | 0.8765 | 0.8513 | 0.9331          | 0.8308 | 0.9848                      | 0.9659 | 0.9679 | 0.9410     | 0.9278 | 0.9247 |
| subcl <sub>IR=7, DR=60</sub>  | 0.9256 | 0.8383 | 0.8727         | 0.8580 | 0.8418 | 0.9376          | 0.8293 | 0.9867                      | 0.9648 | 0.9527 | 0.9269     | 0.9225 | 0.9094 |
| subcl <sub>IR=7, DR=70</sub>  | 0.9263 | 0.8100 | 0.8724         | 0.8490 | 0.8271 | 0.9368          | 0.8137 | 0.9893                      | 0.9405 | 0.9409 | 0.9328     | 0.9339 | 0.8988 |
| clover <sub>IR=5, DR=0</sub>  | 0.9139 | 0.8653 | 0.8908         | 0.8807 | 0.8844 | 0.9024          | 0.8627 | 0.9682                      | 0.9198 | 0.9349 | 0.9181     | 0.9041 | 0.9139 |
| clover <sub>IR=5, DR=30</sub> | 0.8984 | 0.8363 | 0.8833         | 0.8421 | 0.8084 | 0.9207          | 0.8389 | 0.9810                      | 0.9325 | 0.9193 | 0.9255     | 0.9126 | 0.9017 |
| clover <sub>IR=5, DR=50</sub> | 0.8987 | 0.8251 | 0.8659         | 0.8237 | 0.8291 | 0.9327          | 0.8189 | 0.9838                      | 0.9295 | 0.9472 | 0.9316     | 0.9118 | 0.9281 |
| clover <sub>IR=5, DR=60</sub> | 0.8970 | 0.8092 | 0.8663         | 0.8060 | 0.7933 | 0.9359          | 0.7997 | 0.9876                      | 0.9216 | 0.9113 | 0.9361     | 0.9229 | 0.8898 |
| clover <sub>IR=5, DR=70</sub> | 0.8980 | 0.8124 | 0.8676         | 0.8061 | 0.8049 | 0.9269          | 0.7950 | 0.9799                      | 0.9398 | 0.9182 | 0.9252     | 0.8968 | 0.8939 |
| clover <sub>IR=7, DR=0</sub>  | 0.9323 | 0.8798 | 0.9175         | 0.8815 | 0.8756 | 0.9057          | 0.8767 | 0.9766                      | 0.9308 | 0.9240 | 0.9317     | 0.9064 | 0.9063 |
| clover <sub>IR=7, DR=30</sub> | 0.9256 | 0.8223 | 0.8928         | 0.8662 | 0.8536 | 0.8959          | 0.8340 | 0.9937                      | 0.9576 | 0.9196 | 0.9162     | 0.9023 | 0.9054 |
| clover <sub>IR=7, DR=50</sub> | 0.9233 | 0.8141 | 0.9035         | 0.8325 | 0.8283 | 0.9008          | 0.8213 | 0.9911                      | 0.9312 | 0.9290 | 0.9236     | 0.9039 | 0.9075 |
| clover <sub>IR=7, DR=60</sub> | 0.9251 | 0.8055 | 0.8605         | 0.8226 | 0.8173 | 0.9166          | 0.8109 | 0.9864                      | 0.9127 | 0.9246 | 0.9431     | 0.8977 | 0.8956 |
| clover <sub>IR=7, DR=70</sub> | 0.9239 | 0.8073 | 0.8742         | 0.8076 | 0.8072 | 0.8990          | 0.7967 | 0.9895                      | 0.9119 | 0.9104 | 0.9321     | 0.8834 | 0.8840 |
| paw <sub>IR=5, DR=0</sub>     | 0.9259 | 0.9100 | 0.9203         | 0.9173 | 0.9076 | 0.9279          | 0.9100 | 0.9530                      | 0.9645 | 0.9478 | 0.9247     | 0.9308 | 0.9282 |

**Table 6** (continued)

| Data sets       | ORI    | SMOTE  | Pre-processing |        |        | Post-processing |        |        | Combining both pre and post |        |            |        |        |
|-----------------|--------|--------|----------------|--------|--------|-----------------|--------|--------|-----------------------------|--------|------------|--------|--------|
|                 |        |        | kmeans         | B1     | B2     | ENN             | IPF    | KSIPF  | B1-IPF                      | B2-IPF | kmeans-ENN | B1-ENN | B2-ENN |
| paw IR=5, DR=30 | 0.9076 | 0.8463 | 0.9008         | 0.8294 | 0.8232 | 0.9387          | 0.8458 | 0.9857 | 0.9123                      | 0.9060 | 0.9343     | 0.8952 | 0.8867 |
| paw IR=5, DR=50 | 0.9010 | 0.8510 | 0.9063         | 0.8488 | 0.8485 | 0.9297          | 0.8566 | 0.9880 | 0.9383                      | 0.9793 | 0.9500     | 0.9203 | 0.9306 |
| paw IR=5, DR=60 | 0.8906 | 0.8310 | 0.8778         | 0.8325 | 0.8176 | 0.9329          | 0.8309 | 0.9825 | 0.9388                      | 0.9178 | 0.9262     | 0.9134 | 0.8949 |
| paw IR=5, DR=70 | 0.8998 | 0.8402 | 0.8815         | 0.8514 | 0.8382 | 0.9477          | 0.8513 | 0.9823 | 0.9548                      | 0.9880 | 0.9558     | 0.9365 | 0.9459 |
| paw IR=7, DR=0  | 0.9364 | 0.9147 | 0.9249         | 0.9248 | 0.9161 | 0.9328          | 0.9200 | 0.9740 | 0.9546                      | 0.9495 | 0.9418     | 0.9347 | 0.9393 |
| paw IR=7, DR=30 | 0.9269 | 0.8545 | 0.9056         | 0.8559 | 0.8715 | 0.9173          | 0.8554 | 0.9868 | 0.9120                      | 0.9236 | 0.9422     | 0.8992 | 0.9090 |
| paw IR=7, DR=50 | 0.9245 | 0.8489 | 0.9064         | 0.8586 | 0.8534 | 0.9315          | 0.8495 | 0.9907 | 0.9333                      | 0.9409 | 0.9414     | 0.9132 | 0.9115 |
| paw IR=7, DR=60 | 0.9215 | 0.8273 | 0.8757         | 0.8438 | 0.8233 | 0.9334          | 0.8305 | 0.9940 | 0.9319                      | 0.9315 | 0.9267     | 0.9194 | 0.8974 |
| paw IR=7, DR=70 | 0.9198 | 0.8393 | 0.9059         | 0.8369 | 0.8442 | 0.9419          | 0.8269 | 0.9932 | 0.9345                      | 0.9347 | 0.9415     | 0.9192 | 0.9365 |
| Average         | 0.9210 | 0.8509 | 0.8956         | 0.8609 | 0.8496 | 0.9363          | 0.8509 | 0.9832 | 0.9452                      | 0.9430 | 0.9388     | 0.9247 | 0.9205 |

**Table 7** The AUC results obtained by the SVM classifier on the real-world data sets. (e.g., ORI: without oversampling and kmeans: kmeans-SMOTE.)

| Data sets | ORI           | SMOTE         | kmeans        | B1-SMOTE      | B2-SMOTE      | SMOTE         | SMOTE-ENN     | SMOTE-IPF     | KNNOR         | SyMPProD      | SMOTECDDN     | KSIPF         |
|-----------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| D1        | 0.8254        | 0.8318        | 0.8696        | 0.8152        | 0.8022        | 0.9634        | 0.9634        | 0.8340        | 0.8594        | 0.8836        | 0.9502        | <b>0.9685</b> |
| D2        | 0.7939        | 0.8153        | 0.8728        | 0.7838        | 0.7631        | 0.9331        | 0.9331        | 0.8176        | 0.8416        | 0.8980        | 0.8973        | <b>0.9601</b> |
| D3        | 0.7312        | 0.7450        | 0.8179        | 0.7367        | 0.7269        | 0.8711        | 0.8711        | 0.7662        | 0.8309        | 0.8644        | 0.8290        | <b>0.8943</b> |
| D4        | 0.8035        | 0.7904        | 0.8994        | 0.7959        | 0.7823        | 0.9018        | 0.9018        | 0.7987        | 0.8173        | 0.9025        | 0.8671        | <b>0.9717</b> |
| D5        | 0.9548        | 0.9673        | 0.9834        | 0.9658        | 0.9553        | <b>0.9943</b> | <b>0.9943</b> | 0.9722        | 0.9826        | 0.9854        | 0.9901        | 0.9938        |
| D6        | 0.9493        | 0.9741        | 0.9883        | 0.9817        | 0.9747        | <b>0.9945</b> | <b>0.9945</b> | 0.9810        | 0.9880        | 0.9892        | 0.9910        | 0.9779        |
| D7        | 0.9365        | 0.9933        | 0.9918        | 0.9934        | 0.9880        | <b>0.9998</b> | <b>0.9998</b> | 0.9938        | 0.9925        | 0.9942        | 0.9982        | 0.9997        |
| D8        | 0.9412        | 0.9985        | 0.9970        | 0.9972        | 0.9945        | <b>1.0000</b> | <b>1.0000</b> | 0.9987        | 0.9952        | 0.9959        | <b>1.0000</b> | 0.9998        |
| D9        | 0.7799        | 0.9230        | 0.9498        | 0.9833        | 0.9662        | 0.9721        | 0.9721        | 0.9253        | <b>0.9852</b> | 0.9803        | 0.9520        | 0.9706        |
| D10       | 0.9001        | 0.9527        | 0.9810        | 0.9792        | 0.9802        | 0.9616        | 0.9616        | 0.9520        | 0.9551        | 0.9481        | 0.9598        | <b>0.9865</b> |
| D11       | 0.9003        | 0.9724        | 0.9775        | 0.9914        | 0.9836        | 0.9902        | 0.9902        | 0.9710        | 0.9887        | 0.9899        | 0.9875        | <b>0.9963</b> |
| D12       | 0.9496        | 0.9838        | 0.9917        | 0.9872        | 0.9778        | 0.9915        | 0.9915        | 0.9842        | 0.9937        | 0.9966        | 0.9909        | <b>0.9983</b> |
| D13       | 0.9998        | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | 0.9998        | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | 0.9997        | <b>1.0000</b> |
| D14       | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> | <b>1.0000</b> |
| D15       | 0.9873        | 0.9901        | 0.9903        | 0.9861        | 0.9829        | 0.9984        | 0.9984        | 0.9903        | 0.9906        | <b>0.9987</b> | 0.9971        | 0.9979        |
| D16       | 0.8844        | 0.9977        | 0.9986        | 0.9983        | 0.9965        | 0.9992        | 0.9992        | 0.9976        | 0.9985        | 0.9985        | 0.9988        | <b>1.0000</b> |
| D17       | 0.5144        | 0.8750        | 0.8738        | 0.9674        | <b>0.9685</b> | 0.9374        | 0.9374        | 0.8785        | 0.9314        | 0.9340        | 0.9056        | 0.8950        |
| D18       | 0.6106        | 0.9077        | 0.9456        | <b>0.9915</b> | 0.9819        | 0.9569        | 0.9569        | 0.9077        | 0.9524        | 0.9877        | 0.9472        | 0.9620        |
| D19       | 0.8995        | 0.9923        | 0.9926        | 0.9988        | 0.9973        | 0.9986        | 0.9986        | 0.9923        | 0.9981        | 0.9970        | 0.9982        | <b>0.9991</b> |
| D20       | 0.6117        | 0.9746        | 0.9846        | 0.5040        | 0.5019        | 0.9808        | 0.9808        | 0.9746        | 0.9852        | <b>0.9958</b> | 0.9810        | 0.9824        |
| Average   | 0.8487        | 0.9343        | 0.9553        | 0.9228        | 0.9162        | 0.9722        | 0.9722        | 0.9368        | 0.9543        | 0.9670        | 0.9620        | 0.9777        |

**Table 8** The F1-measure results obtained by the SVM classifier on real-world data sets. (e.g., ORI: without oversampling and kmeans: kmeans-SMOTE.)

| Data sets      | ORI    | SMOTE  | kmeans | B1-SMOTE      | B2-SMOTE | SMOTE-SMOTE   | SMOTE-ENN | SMOTE-IPF     | KNNOR         | SyMPProD      | SMOTECDDNN    | KSIPIF        |
|----------------|--------|--------|--------|---------------|----------|---------------|-----------|---------------|---------------|---------------|---------------|---------------|
| D1             | 0.5954 | 0.7387 | 0.7889 | 0.7562        | 0.7404   | <b>0.9251</b> | 0.7344    | 0.7550        | 0.8024        | 0.9070        | 0.9079        | 0.9079        |
| D2             | 0.4021 | 0.7394 | 0.7922 | 0.7510        | 0.7401   | <b>0.8876</b> | 0.7507    | 0.7712        | 0.8279        | 0.8430        | 0.8855        | 0.8855        |
| D3             | 0.1255 | 0.5758 | 0.7108 | 0.6129        | 0.5160   | 0.7630        | 0.6177    | 0.7534        | <b>0.8315</b> | 0.6667        | 0.7647        | 0.7647        |
| D4             | 0.0000 | 0.6803 | 0.8181 | 0.7203        | 0.7087   | 0.8408        | 0.6931    | 0.6883        | 0.8182        | 0.8239        | <b>0.9275</b> | <b>0.9275</b> |
| D5             | 0.7992 | 0.9122 | 0.9373 | 0.9259        | 0.9244   | <b>0.9836</b> | 0.9159    | 0.9472        | 0.9477        | 0.9672        | 0.9596        | 0.9596        |
| D6             | 0.0000 | 0.9062 | 0.9467 | <b>0.9737</b> | 0.9731   | 0.9416        | 0.9096    | 0.9456        | 0.9501        | 0.9324        | 0.9588        | 0.9588        |
| D7             | 0.7957 | 0.9489 | 0.9815 | 0.9759        | 0.9631   | 0.9912        | 0.9671    | 0.9818        | 0.9844        | 0.9809        | <b>0.9930</b> | <b>0.9930</b> |
| D8             | 0.8278 | 0.9830 | 0.9866 | 0.9885        | 0.9808   | <b>0.9988</b> | 0.9842    | 0.9881        | 0.9875        | 0.9943        | 0.9969        | 0.9969        |
| D9             | 0.0686 | 0.8568 | 0.8877 | 0.9412        | 0.9192   | 0.9182        | 0.8526    | 0.9486        | <b>0.9499</b> | 0.8998        | 0.9117        | 0.9117        |
| D10            | 0.5715 | 0.8077 | 0.9774 | 0.9395        | 0.9670   | 0.8098        | 0.8135    | 0.6835        | 0.9262        | 0.8076        | <b>0.9805</b> | <b>0.9805</b> |
| D11            | 0.1253 | 0.9125 | 0.9342 | 0.9591        | 0.9385   | 0.9530        | 0.9093    | 0.9485        | 0.9618        | 0.9417        | <b>0.9667</b> | <b>0.9667</b> |
| D12            | 0.6014 | 0.9547 | 0.9715 | 0.9718        | 0.9572   | 0.9706        | 0.9533    | 0.9764        | 0.9811        | 0.9677        | <b>0.9818</b> | <b>0.9818</b> |
| D13            | 0.9526 | 0.9984 | 0.9989 | 0.9987        | 0.9961   | 0.9981        | 0.9988    | 0.9986        | 0.9989        | <b>0.9993</b> | 0.9990        | 0.9990        |
| D14            | 0.9560 | 0.9998 | 0.9990 | 0.9990        | 0.9959   | 0.9994        | 0.9994    | <b>1.0000</b> | <b>1.0000</b> | 0.9994        | 0.9998        | 0.9998        |
| D15            | 0.3982 | 0.9768 | 0.9765 | 0.9780        | 0.9735   | <b>0.9890</b> | 0.9772    | 0.9798        | 0.9865        | 0.9870        | 0.9860        | 0.9860        |
| D16            | 0.2067 | 0.9729 | 0.9911 | 0.9945        | 0.9865   | 0.9820        | 0.9705    | 0.9944        | 0.9943        | 0.9782        | <b>0.9963</b> | <b>0.9963</b> |
| D17            | 0.0000 | 0.8068 | 0.7579 | <b>0.9359</b> | 0.9288   | 0.9071        | 0.8151    | 0.8765        | 0.9118        | 0.8469        | 0.8116        | 0.8116        |
| D18            | 0.0000 | 0.8555 | 0.8836 | 0.9743        | 0.9572   | 0.8979        | 0.8547    | 0.9079        | <b>0.9750</b> | 0.8983        | 0.9030        | 0.9030        |
| D19            | 0.0000 | 0.9681 | 0.9768 | <b>0.9923</b> | 0.9864   | 0.9792        | 0.9655    | 0.9888        | 0.9860        | 0.9764        | 0.9869        | 0.9869        |
| D20            | 0.0000 | 0.8994 | 0.9044 | 0.0000        | 0.0000   | 0.9288        | 0.8971    | 0.9166        | <b>0.9945</b> | 0.9192        | 0.9225        | 0.9225        |
| <b>Average</b> | 0.3713 | 0.8747 | 0.9110 | 0.8694        | 0.8576   | 0.9332        | 0.8790    | 0.9025        | 0.9408        | 0.9168        | 0.9420        | 0.9420        |



**Table 9** The AUC results obtained by the AdaBoost classifier on the real-world data sets. (e.g., ORI: without oversampling and kmeans: kmeans-SMOTE.)

| Data sets      | ORI    | SMOTE         | kmeans | B1-SMOTE | B2-SMOTE | SMOTE         | SMOTE-ENN     | SMOTE-IPF     | KNNOR         | SyMPProD | SMOTECDDN     | KSIPF |
|----------------|--------|---------------|--------|----------|----------|---------------|---------------|---------------|---------------|----------|---------------|-------|
| D1             | 0.8103 | 0.8552        | 0.8681 | 0.8491   | 0.8328   | 0.852         | 0.8530        | 0.8776        | 0.8926        | 0.9574   | <b>0.9781</b> |       |
| D2             | 0.7915 | 0.8514        | 0.8854 | 0.8428   | 0.8103   | 0.9537        | 0.8489        | 0.8966        | 0.9059        | 0.9177   | <b>0.9767</b> |       |
| D3             | 0.6273 | 0.7670        | 0.8114 | 0.7864   | 0.7490   | 0.8994        | 0.7863        | 0.8352        | 0.8537        | 0.8807   | <b>0.9378</b> |       |
| D4             | 0.8389 | 0.9020        | 0.9342 | 0.9072   | 0.8879   | 0.9741        | 0.9071        | 0.9248        | 0.9387        | 0.9621   | <b>0.9859</b> |       |
| D5             | 0.9291 | 0.9592        | 0.9676 | 0.9510   | 0.9451   | 0.9896        | 0.9578        | 0.9643        | 0.9656        | 0.9801   | <b>0.9941</b> |       |
| D6             | 0.9426 | 0.9925        | 0.9893 | 0.9925   | 0.9927   | <b>0.9994</b> | 0.9921        | 0.9914        | 0.9949        | 0.9944   | 0.9974        |       |
| D7             | 0.9325 | 0.9873        | 0.9957 | 0.9933   | 0.9891   | 0.9991        | 0.9884        | 0.9965        | 0.9957        | 0.9969   | <b>1.0000</b> |       |
| D8             | 0.9517 | 0.9958        | 0.9968 | 0.9952   | 0.9940   | <b>0.9987</b> | 0.9934        | 0.9963        | 0.9977        | 0.9981   | 0.9971        |       |
| D9             | 0.7643 | 0.9541        | 0.9735 | 0.9749   | 0.9709   | 0.9774        | 0.9510        | 0.9757        | 0.9740        | 0.9626   | <b>0.9914</b> |       |
| D10            | 0.9992 | 0.9989        | 0.9999 | 0.9997   | 0.9998   | 0.9988        | 0.9989        | 0.9988        | 0.9989        | 0.9988   | <b>0.9999</b> |       |
| D11            | 0.7971 | 0.9546        | 0.9724 | 0.9817   | 0.9794   | 0.9789        | 0.9557        | 0.9850        | 0.9838        | 0.9763   | <b>0.9958</b> |       |
| D12            | 0.9557 | 0.9955        | 0.9958 | 0.9964   | 0.9954   | 0.9978        | 0.9961        | 0.9972        | 0.9971        | 0.9970   | <b>0.9998</b> |       |
| D13            | 0.9983 | 0.9996        | 0.9986 | 0.9994   | 0.9998   | 0.9996        | 0.9996        | <b>0.9999</b> | <b>0.9999</b> | 0.9997   | 0.9998        |       |
| D14            | 0.9865 | <b>0.9998</b> | 0.9994 | 0.9992   | 0.9994   | 0.9996        | <b>0.9998</b> | <b>0.9998</b> | 0.9992        | 0.9994   | <b>0.9998</b> |       |
| D15            | 0.9845 | 0.9942        | 0.9947 | 0.9925   | 0.9927   | 0.9988        | 0.9946        | 0.9968        | 0.9984        | 0.9987   | <b>0.9999</b> |       |
| D16            | 0.8185 | 0.9924        | 0.9940 | 0.9942   | 0.9939   | 0.9959        | 0.9905        | 0.9931        | 0.9938        | 0.9943   | <b>1.0000</b> |       |
| D17            | 0.7689 | 0.9808        | 0.9918 | 0.9930   | 0.9923   | 0.9819        | 0.9820        | 0.9936        | 0.9911        | 0.9868   | <b>0.9987</b> |       |
| D18            | 0.6977 | 0.9370        | 0.9878 | 0.9926   | 0.9936   | 0.9719        | 0.9406        | 0.9865        | 0.9921        | 0.9656   | <b>0.9958</b> |       |
| D19            | 0.8778 | 0.9930        | 0.9967 | 0.9978   | 0.9969   | 0.9974        | 0.9924        | 0.9976        | 0.9973        | 0.9979   | <b>0.9995</b> |       |
| D20            | 0.5616 | 0.9946        | 0.9949 | 0.5618   | 0.5478   | 0.9949        | 0.9947        | 0.9946        | 0.9943        | 0.9949   | <b>1.0000</b> |       |
| <b>Average</b> | 0.8517 | 0.9552        | 0.9674 | 0.9400   | 0.9331   | 0.9846        | 0.9561        | 0.9701        | 0.9732        | 0.9780   | 0.9924        |       |

**Table 10** The F1-measure results obtained by the AdaBoost classifier on real-world data sets. (e.g., ORI: without oversampling and kmeans: kmeans-SMOTE.)

| Data sets      | ORI    | SMOTE         | kmeans | B1-SMOTE | B2-SMOTE | SMOTE         | SMOTE-ENN     | SMOTE-IPF     | KNNOR         | SyMPProD | SMOTECDDN     | KSIPIF        |
|----------------|--------|---------------|--------|----------|----------|---------------|---------------|---------------|---------------|----------|---------------|---------------|
| D1             | 0.6091 | 0.7801        | 0.7921 | 0.7936   | 0.7639   | 0.9491        | 0.9491        | 0.7786        | 0.8088        | 0.8028   | 0.9039        | <b>0.9243</b> |
| D2             | 0.5344 | 0.7751        | 0.7989 | 0.7772   | 0.7543   | 0.8999        | 0.8999        | 0.7811        | 0.8269        | 0.8211   | 0.8559        | <b>0.9234</b> |
| D3             | 0.3537 | 0.7146        | 0.7420 | 0.7382   | 0.6936   | 0.8393        | 0.8393        | 0.7256        | 0.7780        | 0.8117   | 0.8221        | <b>0.8719</b> |
| D4             | 0.5146 | 0.8289        | 0.8504 | 0.8368   | 0.8169   | 0.9478        | 0.9478        | 0.8310        | 0.8465        | 0.8513   | 0.9256        | <b>0.9524</b> |
| D5             | 0.7700 | 0.9018        | 0.9154 | 0.9153   | 0.9117   | <b>0.9757</b> | <b>0.9757</b> | 0.9169        | 0.9381        | 0.9322   | 0.9650        | 0.9725        |
| D6             | 0.8502 | 0.9764        | 0.9811 | 0.9762   | 0.9783   | <b>0.9905</b> | <b>0.9905</b> | 0.9799        | 0.9793        | 0.9804   | 0.9871        | 0.9903        |
| D7             | 0.6537 | 0.9538        | 0.9804 | 0.9756   | 0.9675   | 0.9860        | 0.9860        | 0.9639        | 0.9768        | 0.9745   | 0.9795        | <b>1.0000</b> |
| D8             | 0.7260 | 0.9824        | 0.9843 | 0.9813   | 0.9792   | 0.9914        | 0.9914        | 0.9806        | 0.9793        | 0.9818   | <b>0.9919</b> | 0.9918        |
| D9             | 0.2424 | 0.8980        | 0.9295 | 0.9498   | 0.9417   | 0.9538        | 0.9538        | 0.8934        | 0.9549        | 0.9453   | 0.9256        | <b>0.9649</b> |
| D10            | 0.9387 | 0.9967        | 0.9971 | 0.9962   | 0.9953   | 0.9964        | 0.9964        | 0.9962        | 0.9953        | 0.9962   | 0.9971        | <b>0.9973</b> |
| D11            | 0.4274 | 0.8975        | 0.9308 | 0.9496   | 0.9516   | 0.9496        | 0.9496        | 0.8967        | 0.9680        | 0.9591   | 0.9342        | <b>0.9833</b> |
| D12            | 0.7577 | 0.9771        | 0.9808 | 0.9824   | 0.9779   | 0.9846        | 0.9846        | 0.9798        | 0.9896        | 0.9875   | 0.9786        | <b>0.9963</b> |
| D13            | 0.8744 | 0.9954        | 0.9949 | 0.9958   | 0.9950   | 0.9985        | 0.9985        | 0.9959        | 0.9945        | 0.9951   | <b>0.9990</b> | 0.9987        |
| D14            | 0.9783 | <b>0.9998</b> | 0.9994 | 0.9992   | 0.9994   | 0.9996        | 0.9996        | <b>0.9998</b> | <b>0.9998</b> | 0.9992   | 0.9994        | <b>0.9998</b> |
| D15            | 0.6512 | 0.9860        | 0.9875 | 0.9869   | 0.9819   | 0.9967        | 0.9967        | 0.9864        | 0.9880        | 0.9906   | 0.9965        | <b>0.9987</b> |
| D16            | 0.4583 | 0.9758        | 0.9914 | 0.9901   | 0.9877   | 0.9872        | 0.9872        | 0.9715        | 0.9901        | 0.9903   | 0.9812        | <b>1.0000</b> |
| D17            | 0.1570 | 0.9580        | 0.9757 | 0.9825   | 0.9812   | 0.9698        | 0.9698        | 0.9569        | 0.9769        | 0.9758   | 0.9635        | <b>0.9906</b> |
| D18            | 0.0189 | 0.8719        | 0.9484 | 0.9728   | 0.9801   | 0.9352        | 0.9352        | 0.8762        | 0.9486        | 0.9752   | 0.9203        | <b>0.9839</b> |
| D19            | 0.3399 | 0.9717        | 0.9846 | 0.9921   | 0.9891   | 0.9858        | 0.9858        | 0.9694        | 0.9904        | 0.9893   | 0.9865        | <b>0.9967</b> |
| D20            | 0.0000 | 0.9818        | 0.9929 | 0.0000   | 0.0000   | 0.9788        | 0.9788        | 0.9782        | 0.9929        | 0.9912   | 0.9796        | <b>1.0000</b> |
| <b>Average</b> | 0.5428 | 0.9211        | 0.9379 | 0.8896   | 0.8823   | 0.9658        | 0.9658        | 0.9229        | 0.9461        | 0.9475   | 0.9546        | 0.9769        |

Sensitivity, which reflects the effectiveness of the algorithm in identifying real minority samples. If and only when both Precision and Recall reach the highest value, F1-measure will reach the highest value. According to [37], adjust the value of the parameter to  $n = 9$ , the number of iterations is chosen as  $T = 3$  and  $p = 1\%$ . The results of KSIPF and other compared methods are obtained by using a tenfold cross-validation procedure. In addition, our experimental environment is based on Python 3.10, where the oversampling method in `imblearn.over_sampling` is used.

Furthermore, the Wilcoxon signed-ranks statistical test is applied to evaluate the suggested filter method. The basic idea of the Wilcoxon test is to sort the absolute value of the difference between two groups of samples from smallest to largest to get the corresponding rank, add them up according to different signs to get the test statistic, and finally find the distribution table to get the P-value. If the P-value is small enough, it means that there is a significant difference between the two samples, and usually, the threshold P-value is 0.05. Besides,  $R^+$  is the strength of the proposed method, and  $R^-$  shows its weakness. In addition, the Friedman test [51] and the Holm test [52] are also used to compare the performance of different oversampling techniques. Friedman test, or Friedman two-way rank analysis of variance, is a statistical test for the homogeneity of multiple samples. Friedman inspection is defined as follows:

$$\chi_F^2 = \frac{12N}{k(k+1)} \left( \sum_{i=1}^k r_i^2 - \frac{k(k+1)^2}{4} \right), \quad (13)$$

where  $N$  is the number of data sets,  $k$  is the number of algorithms, and  $r_i$  is the average ranking of the  $i$ th algorithm. The experimental process is composed of four steps as shown in Fig. 3.

## 5 Results and analysis

### 5.1 Comparison with SMOTE and its variants

The results of 13 different algorithms on 30 synthetic data sets and 11 different algorithms on 20 real-world data sets are shown in Tables 3, 4, 5, 6, 7, 8, 9, and 10, and the best results are shown in bold. The SVM and AdaBoost classifiers are used to evaluate the performance of the algorithm on AUC and F1-measure, respectively. The first column indexed by "ORI" represents the result without pre-processing. The next columns are the results of oversampling obtained by SMOTE and its variants. KSIPF is the method proposed in this paper, and the remaining columns are the results of the combination of pre-processing and post-processing. From the data in Tables 3, 4, 5, 6, 7, 8, 9, and 10, the following conclusions can be drawn.

1. First of all, SMOTE and its variants show improvements in imbalanced classification, whether on synthetic data sets or real-world data sets. This fully shows that SMOTE and its variants are effective oversampling techniques, which can

**Table 11** Wilcoxon's test results of the SVM classifier on the synthetic data sets

| Metrics    | Methods             | $R^+$ | $R^-$ | $P_{\text{Wilcoxon}}$ |
|------------|---------------------|-------|-------|-----------------------|
| AUC        | KSIPF vs. ORI       | 465.0 | 0.0   | 0.000002              |
|            | KSIPF vs. SMOTE     | 465.0 | 0.0   | 0.000002              |
|            | KSIPF vs. SMOTE-ENN | 262.0 | 203.0 | 0.543996              |
|            | KSIPF vs. B1-IPF    | 240.0 | 225.0 | 0.877400              |
| F1-measure | KSIPF vs. ORI       | 465.0 | 0.0   | 0.000002              |
|            | KSIPF vs. SMOTE     | 465.0 | 0.0   | 0.000002              |
|            | KSIPF vs. SMOTE-ENN | 462.0 | 3.0   | 0.000002              |
|            | KSIPF vs. B1-IPF    | 465.0 | 0.0   | 0.000002              |

**Table 12** Wilcoxon's test results of the AdaBoost classifier on the synthetic data sets

| Metrics    | Methods             | $R^+$ | $R^-$ | $P_{\text{Wilcoxon}}$ |
|------------|---------------------|-------|-------|-----------------------|
| AUC        | KSIPF vs. ORI       | 462.0 | 3.0   | 0.000002              |
|            | KSIPF vs. SMOTE     | 459.0 | 6.0   | 0.000003              |
|            | KSIPF vs. SMOTE-ENN | 292.5 | 172.5 | 0.217153              |
|            | KSIPF vs. B1-IPF    | 243.5 | 221.5 | 0.820996              |
| F1-measure | KSIPF vs. ORI       | 462.0 | 3.0   | 0.000002              |
|            | KSIPF vs. SMOTE     | 462.0 | 3.0   | 0.000002              |
|            | KSIPF vs. SMOTE-ENN | 460.0 | 5.0   | 0.000003              |
|            | KSIPF vs. B1-IPF    | 456.0 | 9.0   | 0.000004              |

**Table 13** Statistical analysis results of synthetic data sets at the confidence level of  $\alpha=0.05$  based on SVM classifier

| Algorithm         | AUC     |              | F1-measure |              |
|-------------------|---------|--------------|------------|--------------|
|                   | Ranking | Hypothesis   | Ranking    | Hypothesis   |
| KSIPF             | 3.18    | -            | 1.10       | -            |
| ORI               | 12.90   | Rejected     | 5.17       | Rejected     |
| SMOTE             | 9.90    | Rejected     | 11.27      | Rejected     |
| kmeans-SMOTE      | 7.17    | Rejected     | 8          | Rejected     |
| Borderline-SMOTE1 | 9.80    | Rejected     | 10.93      | Rejected     |
| Borderline-SMOTE2 | 12.07   | Rejected     | 12.97      | Rejected     |
| SMOTE-ENN         | 3.30    | Not rejected | 3.2        | Not rejected |
| SMOTE-IPF         | 9.92    | Rejected     | 10.73      | Rejected     |
| Borderline1-IPF   | 3.23    | Not rejected | 5.4        | Rejected     |
| Borderline2-IPF   | 4.97    | Not rejected | 7.5        | Rejected     |
| kmeans-ENN        | 4.37    | Not rejected | 3.07       | Not rejected |
| Borderline1-ENN   | 4.23    | Not rejected | 4.87       | Rejected     |
| Borderline2-ENN   | 5.97    | Not rejected | 6.8        | Rejected     |

**Table 14** Statistical analysis results of synthetic data sets at the confidence level of  $\alpha=0.05$  based on AdaBoost classifier

| Algorithm         | AUC     |              | F1-measure |              |
|-------------------|---------|--------------|------------|--------------|
|                   | Ranking | Hypothesis   | Ranking    | Hypothesis   |
| KSIPF             | 3.60    | –            | 1.60       | –            |
| ORI               | 12.9    | Rejected     | 6.12       | Rejected     |
| SMOTE             | 10.2    | Rejected     | 11.28      | Rejected     |
| kmeans-SMOTE      | 7.02    | Rejected     | 8.93       | Rejected     |
| Borderline-SMOTE1 | 9.42    | Rejected     | 10.3       | Rejected     |
| Borderline-SMOTE2 | 10.7    | Rejected     | 11.9       | Rejected     |
| SMOTE-ENN         | 4.53    | Not rejected | 4.43       | Not rejected |
| SMOTE-IPF         | 10.13   | Rejected     | 11.28      | Rejected     |
| Borderline1-IPF   | 3.12    | Not rejected | 3.4        | Not rejected |
| Borderline2-IPF   | 3.17    | Not rejected | 3.93       | Not rejected |
| kmeans-ENN        | 4.97    | Not rejected | 4.23       | Not rejected |
| Borderline1-ENN   | 5.4     | Not rejected | 6.5        | Rejected     |
| Borderline2-ENN   | 5.85    | Not rejected | 7.08       | Rejected     |

**Table 15** Wilcoxon's test results of the SVM classifier on the real-world data sets

| Metrics    | Methods                | R <sup>+</sup> | R <sup>−</sup> | P <sub>Wilcoxon</sub> |
|------------|------------------------|----------------|----------------|-----------------------|
| AUC        | KSIPF vs. ORI          | 190.0          | 0.0            | 0.000132              |
|            | KSIPF vs. SMOTE        | 171.0          | 0.0            | 0.000196              |
|            | KSIPF vs. kmeans-SMOTE | 159.5          | 11.5           | 0.001268              |
|            | KSIPF vs. SyMProD      | 113.0          | 58.0           | 0.231059              |
|            | KSIPF vs. SMOTECDDN    | 166.0          | 24.0           | 0.004274              |
| F1-measure | KSIPF vs. ORI          | 210.0          | 0.0            | 0.000089              |
|            | KSIPF vs. SMOTE        | 190.0          | 0.0            | 0.000132              |
|            | KSIPF vs. kmeans-SMOTE | 210.0          | 0.0            | 0.000088              |
|            | KSIPF vs. SyMProD      | 127.0          | 83.0           | 0.411424              |
|            | KSIPF vs. SMOTECDDN    | 181.0          | 29.0           | 0.004550              |

improve the performance of classifiers on imbalanced data. However, these oversampling techniques are often only improved for some aspects and cannot be applied to all data sets. Therefore, oversampling techniques still need to be further studied and improved.

- Compared with some SMOTE variant methods, the original SMOTE method has achieved good performance. Although the SMOTE method has some defects and deviations in generating new samples. However, it still increases the number of samples of minority, making the data set into balanced data. To a certain extent, this solves the defect of traditional classifiers in dealing with unbalanced data. In particular, it has excellent performance on some data sets such as “winequality\_3vs\_5” and “abalone19,” which is greatly improved compared to

**Table 16** Wilcoxon's test results of the AdaBoost classifier on the real-world data sets

| Metrics    | Methods                | R <sup>+</sup> | R <sup>-</sup> | P <sub>Wilcoxon</sub> |
|------------|------------------------|----------------|----------------|-----------------------|
| AUC        | KSIPF vs. ORI          | 210.0          | 0.0            | 0.000089              |
|            | KSIPF vs. SMOTE        | 190.0          | 0.0            | 0.000132              |
|            | KSIPF vs. kmeans-SMOTE | 190.0          | 0.0            | 0.000132              |
|            | KSIPF vs. SyMProD      | 206.5          | 3.5            | 0.000151              |
|            | KSIPF vs. SMOTECDDN    | 207.0          | 3.0            | 0.000140              |
| F1-measure | KSIPF vs. ORI          | 210.0          | 0.0            | 0.000089              |
|            | KSIPF vs. SMOTE        | 190.0          | 0.0            | 0.000132              |
|            | KSIPF vs. kmeans-SMOTE | 210.0          | 0.0            | 0.000089              |
|            | KSIPF vs. SyMProD      | 210.0          | 0.0            | 0.000088              |
|            | KSIPF vs. SMOTECDDN    | 206.0          | 4.0            | 0.000163              |

**Table 17** Statistical analysis results of real-world data sets at the confidence level of  $\alpha=0.05$  based on SVM classifier

| Algorithm         | AUC     |              | F1-measure |              |
|-------------------|---------|--------------|------------|--------------|
|                   | Ranking | Hypothesis   | Ranking    | Hypothesis   |
| KSIPF             | 2.88    | —            | 2.73       | —            |
| ORI               | 10.03   | Rejected     | 10.95      | Rejected     |
| SMOTE             | 8.10    | Rejected     | 8.73       | Rejected     |
| kmeans-SMOTE      | 5.85    | Not rejected | 6.15       | Not rejected |
| Borderline-SMOTE1 | 6.13    | Not rejected | 4.83       | Not rejected |
| Borderline-SMOTE2 | 8.33    | Rejected     | 7.00       | Rejected     |
| SMOTE-ENN         | 3.10    | Not rejected | 4.10       | Not rejected |
| SMOTE-IPF         | 7.43    | Rejected     | 8.30       | Rejected     |
| KNNOR             | 5.25    | Not rejected | 4.93       | Not rejected |
| SyMProD           | 4.05    | Not rejected | 3.10       | Not rejected |
| SMOTECDDN         | 4.88    | Not rejected | 5.20       | Not rejected |

without oversampling. In other cases, the effect of SMOTE on both data sets is significant.

3. In the synthetic data sets, the proposed KSIPF has achieved an absolute advantage. Under the SVM classifier, it obtained 20 and 29 optimal results, and under the AdaBoost classifier, it achieved 18 and 26 optimal results, which fully demonstrates the efficiency and significance of KSIPF. On the contrary, the method combining Borderline-SMOTE with IPF only yields good results when AUC is used as the evaluation metric, but fails to perform well under the F1-measure. This further highlights the robustness of KSIPF, as it consistently delivers superior performance across different metrics and classifiers. In this context, maximizing True-Positive (TP) outcomes is crucial, as it represents the best scenario, while minimizing False-Negative (FN) cases is equally important, as it reflects the least desirable scenario. The strong performance of KSIPF in achieving high TP rates

**Table 18** Statistical analysis results of real-world data sets at the confidence level of  $\alpha=0.05$  based on AdaBoost classifier

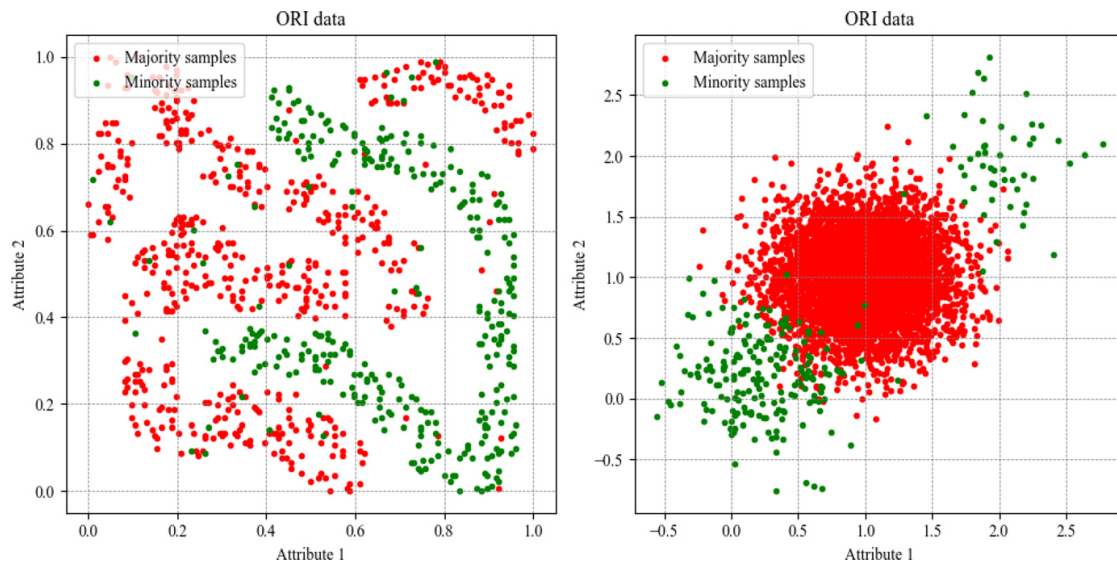
| Algorithm         | AUC     |              | F1-measure |              |
|-------------------|---------|--------------|------------|--------------|
|                   | Ranking | Hypothesis   | Ranking    | Hypothesis   |
| KSIPF             | 1.48    | —            | 1.33       | —            |
| ORI               | 10.65   | Rejected     | 10.95      | Rejected     |
| SMOTE             | 7.98    | Rejected     | 8.03       | Rejected     |
| kmeans-SMOTE      | 6.00    | Rejected     | 5.60       | Rejected     |
| Borderline-SMOTE1 | 6.55    | Rejected     | 6.23       | Rejected     |
| Borderline-SMOTE2 | 7.33    | Rejected     | 7.73       | Rejected     |
| SMOTE-ENN         | 3.65    | Not rejected | 3.83       | Not rejected |
| SMOTE-IPF         | 7.93    | Rejected     | 7.83       | Rejected     |
| KNNOR             | 4.98    | Not rejected | 4.80       | Not rejected |
| SymProD           | 4.88    | Not rejected | 4.93       | Not rejected |
| SMOTECDDN         | 4.60    | Not rejected | 4.78       | Not rejected |

and reducing FN instances underscores its effectiveness in handling imbalanced data sets.

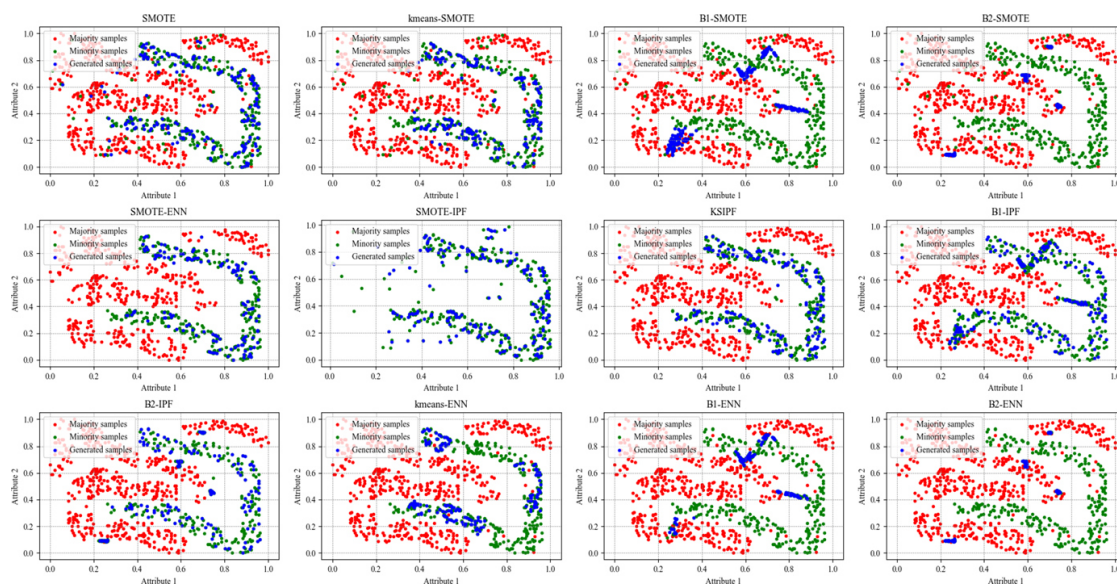
4. In real-world data sets, KSIPF achieved better results in comparison with SMOTE and its variant methods. From the results in Tables 7, 8, 9, and 10, we can see that the average value obtained on the 20 data sets is always the highest, and the number of times the best results are obtained is also the largest. All of these show that KSIPF is an effective oversampling technique.
5. In addition, it is worth noting that all "combining both pre and post" methods have achieved better results than before. Among them, the technology after the combination of Borderline-SMOTE1 and IPF has been significantly improved. Even getting the best results on real data sets is more frequent than KSIPF. This shows that the method proposed in this paper is effective for the proposed KSIPF, and significant for other oversampling techniques. To some extent, this combination can improve the performance of oversampling technology in the face of imbalanced data.

## 5.2 Statistical test on data sets

To better compare different algorithms, the results of nonparametric hypothesis testing are shown in Tables 11, 12, 13, 14, 15, 16, 17, and 18. In each group, the null hypothesis is that there is no significant difference between the two groups ( $H_0$  = no significant difference between the two groups results). First of all, there are the results of the Wilcoxon test, in which KSIPF is compared with ORI, SMOTE, and two better oversampling techniques SMOTE-ENN and B1-IPF in Tables 11



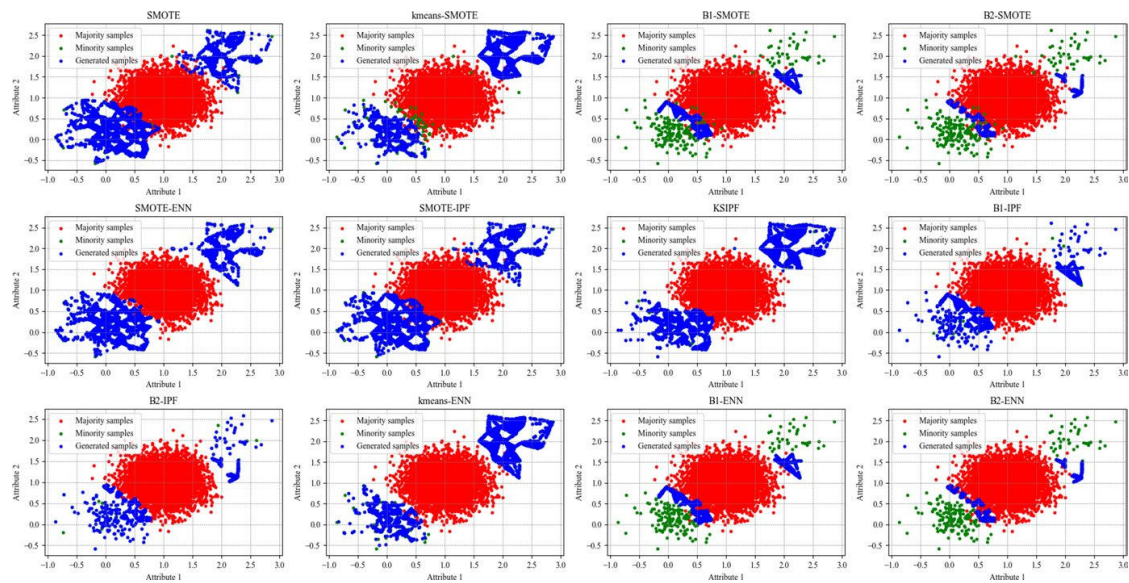
**Fig. 4** Two types of data distribution: The left one is Fourclass, and the right one is Gaussian



**Fig. 5** Fourclass data distribution after processing by SMOTE and various SMOTE variants

and 15, respectively. As mentioned earlier,  $R^+$  and  $R^-$  represent the advantages and disadvantages of KSIPF, respectively. The larger the  $R^+$  is, the better the proposed oversampling technique is. For example, in Table 11, the result of "KSIPF vs. ORI" indicates that the advantage ( $R^+$ ) of KSIPF is far from its disadvantage ( $R^-$ ) ( $465.0 - 0.0 = 456.0$ ). In synthetic data sets, KSIPF has a great advantage over the comparative oversampling technique, and its  $R^+$  is always larger than  $R^-$ . When AdaBoost is the classifier, it has more obvious advantages, in which all P values are less than 0.05, which means that the proposed oversampling technique has a significant advantage over the comparative technique. KSIPF also has a higher  $R^+$  on





**Fig. 6** Gaussian data distribution after processing by SMOTE and various SMOTE variants

real data sets, except when compared to SMOTE-ENN. Generally speaking, KSIPF shows superiority and high efficiency in the Wilcoxon test.

The second is the Friedman test, as shown in Tables 13, 14, 17, and 18. In the experiment, KSIPF is used as a control method and compared with other oversampling techniques. The ranking under AUC and F1-measure and whether to reject the original hypothesis are given in the tables. Similarly, KSIPF achieves the highest rank on both synthetic and real data sets. Specifically, in the synthetic data sets, KSIPF achieved a rank of 3.18, 1.10, 3.60, and 1.60, respectively, and passed the hypothesis test in most of the comparison algorithms. In the real-world data sets, KSIPF also achieved the highest rank, 2.88, 2.73, 1.48, and 1.33, respectively. However, it does not pass the hypothesis test well. Obviously, the two oversampling techniques of SMOTE-ENN and SMOTECDDN also achieved excellent results, which is the reason why they were selected for the Wilcoxon test. However, it is worth noting that the proposed KSIPF gets the best rank every time, which shows that KSIPF is still an efficient oversampling technique even if it does not pass the hypothesis test.

### 5.3 Visualization of algorithms

To reflect the difference between KSIPF and other oversampling techniques, comparative experiments are carried out on two synthetic data sets. The details of the two synthetic data sets are as follows: The first data set is generated by Gaussian distribution, in which the mean of the majority samples is 1, the variance is 0.1, and the minority samples are divided into two parts, the mean is 0.2 and 2, respectively, and the variance is 0.1. The second data set is obtained from the classical “fourclass” data set, in which the sample label is converted into another kind of label with a probability of 0.05 to generate some noise.

First of all, the original distribution of two synthetic data sets is given, as shown in Fig. 4. Then, it shows the data distribution produced by different oversampling techniques, including SMOTE, Borderline-SMOTE1, Borderline-SMOTE2, kmeans-SMOTE, SMOTE-ENN, SMOTE-IPF, KSIPF, B1-IPF, B2-IPF, kmeans-ENN, B1-ENN, and B2-ENN. The majority samples are represented in red, the minority samples in green, and the newly generated samples in blue, as shown in Figs. 5 and 6.

Figures 5 and 6 show that there are some noise samples in the original data, and SMOTE only achieves a balance in quantity but does not take into account the noise samples. The two kinds of Borderline-SMOTE only generate new samples at the class boundary, but the characteristics of artificially defined boundary samples do not apply to all data sets. In the distribution of more complex data, the resulting data quality is worse. Kmeans-SMOTE avoids the generation of noise samples but does not deal with existing noise samples. SMOTE-IPF performed poorly on fourclass data sets but achieved good oversampling results on the second data set. SMOTE-ENN is relatively good at dealing with noise samples, but in the fourclass data set, we can see that SMOTE-ENN will remove too many samples, and the final data is not balanced. Finally, according to the observation of KSIPF and the rest of the combined oversampling technology, KSIPF has achieved an ideal effect of the two data. KSIPF not only removes the noise samples but also makes the data distribution more reasonable. The performance of the two kinds of synthetic data is very good, indicating that KSIPF is an efficient oversampling method.

## 6 Conclusion and future work

In this paper, we obtained an efficient oversampling method: KSIPF, which combines the advantages of pre-processing and post-processing variants. Firstly, the imbalanced data are clustered into  $k$  classes based on similarity, then select the cluster that the samples of the minority class are dominant for oversampling; finally, the IPF filter is used for noise removal to obtain a balanced data set. Next, SVM and AdaBoost as the final classifiers are used to compare KSIPF with other SMOTE variants, and also with their combination with IPF filter, and find that KSIPF is effective in improving the oversampling performance.

Nonetheless, there are also some limitations in this paper. The oversampling technique proposed in this paper does not use more advanced deep learning methods to improve the quality of synthetic samples and does not provide further improvements to IPF. In the future, more information about the distribution of the data samples (e.g., the distribution density of the samples) will be considered. In addition, the partition classification method in IPF can also be improved, which is also a direction of our future research, to propose a better classification method to replace C4.5 used in this paper. In addition, seeking corresponding application areas in real life is also a topic worthy of research, such as medical diagnosis and risk assessment.

**Author contribution** Pengfei Sun helped in conceptualization, methodology, software, validation, formal analysis, data curation, writing—original draft, writing—review & editing, and visualization. Zhiping Wang helped in conceptualization, methodology, methodology, writing—review & editing, and supervision. Liyan Jia helped in conceptualization, methodology, software, and writing—review & editing. Xiaoxi Wang helped in data curation and supervision.

**Data availability** No data sets were generated or analyzed during the current study.

## Declarations

**Competing interests** The authors declare no competing interests.

## References

1. Lu X, Ma C, Shen J et al (2022) Deep object tracking with shrinkage loss. *IEEE Trans Pattern Anal Mach Intell* 44:2386–2401. <https://doi.org/10.1109/TPAMI.2020.3041332>
2. Bennin KE, Keung J, Phannachitta P et al (2018) MAHAKIL: diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction. *IEEE Trans Softw Eng* 44:534–550. <https://doi.org/10.1109/TSE.2017.2731766>
3. Nasrollahpour H, Isildak I, Rashidi M-R et al (2021) Ultrasensitive bioassaying of HER-2 protein for diagnosis of breast cancer using reduced graphene oxide/chitosan as nanobiocompatible platform. *Cancer Nano* 12:10. <https://doi.org/10.1186/s12645-021-00082-y>
4. Yao P, Shen S, Xu M et al (2022) Single model deep learning on imbalanced small datasets for skin lesion classification. *IEEE Trans Med Imaging* 41:1242–1254. <https://doi.org/10.1109/TMI.2021.3136682>
5. Zhao C, Xin Y, Li X et al (2020) A heterogeneous ensemble learning framework for spam detection in social networks with imbalanced data. *Appl Sci* 10:936. <https://doi.org/10.3390/app10030936>
6. Mahajan PD, Maurya A, Megahed A et al (2020) Optimizing predictive precision in imbalanced datasets for actionable revenue change prediction. *Eur J Oper Res* 285:1095–1113. <https://doi.org/10.1016/j.ejor.2020.02.036>
7. Yeung M, Sala E, Schoenlieb C-B, Rundo L (2022) Unified focal loss: generalising dice and cross entropy-based losses to handle class imbalanced medical image segmentation. *Comput Med Imaging Graph* 95:102026. <https://doi.org/10.1016/j.compmedimag.2021.102026>
8. Hou S, Liu Y, Yang Q (2022) Real-time prediction of rock mass classification based on TBM operation big data and stacking technique of ensemble learning. *J Rock Mech Geotech Eng* 14:123–143. <https://doi.org/10.1016/j.jrmge.2021.05.004>
9. Liu W, Lin H, Huang L, et al (2022) Identification of miRNA-disease associations via deep forest ensemble learning based on autoencoder. *Brief Bioinform* 23. <https://doi.org/10.1093/bib/bbac104>
10. Sun J, Li H, Fujita H et al (2020) Class-imbalanced dynamic financial distress prediction based on Adaboost-SVM ensemble combined with SMOTE and time weighting. *Inf Fusion* 54:128–144. <https://doi.org/10.1016/j.inffus.2019.07.006>
11. Zhang X, Zhuang Y, Wang W, Pedrycz W (2018) Transfer boosting with synthetic instances for class imbalanced object recognition. *IEEE Trans Cybern* 48:357–370. <https://doi.org/10.1109/TCYB.2016.2636370>
12. Yu H, Sun C, Yang X et al (2019) Fuzzy support vector machine with relative density information for classifying imbalanced data. *IEEE Trans Fuzzy Syst* 27:2353–2367. <https://doi.org/10.1109/TFUZZ.2019.2898371>
13. Wang Z, Wang B, Cheng Y et al (2019) Cost-sensitive fuzzy multiple kernel learning for imbalanced problem. *Neurocomputing* 366:178–193. <https://doi.org/10.1016/j.neucom.2019.06.065>
14. Huang Z, Dumitru CO, Pan Z et al (2021) Classification of large-scale high-resolution SAR images with deep transfer learning. *IEEE Geosci Remote Sens Lett* 18:107–111. <https://doi.org/10.1109/LGRS.2020.2965558>
15. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) SMOTE: synthetic minority over-sampling technique. *Jair* 16:321–357. <https://doi.org/10.1613/jair.953>

16. Batista GEAPA, Prati RC, Monard MC (2004) A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explor Newsl* 6:20–29. <https://doi.org/10.1145/1007730.1007735>
17. Douzas G, Bacao F, Last F (2018) Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE. *Inf Sci* 465:1–20. <https://doi.org/10.1016/j.ins.2018.06.056>
18. Zhang A, Yu H, Zhou S et al (2022) Instance weighted SMOTE by indirectly exploring the data distribution. *Knowl-Based Syst* 249:108919. <https://doi.org/10.1016/j.knosys.2022.108919>
19. Li M, Zhou H, Liu Q, Wang G (2022) SW: A weighted space division framework for imbalanced problems with label noise. *Knowl-Based Syst* 251:109233. <https://doi.org/10.1016/j.knosys.2022.109233>
20. Jia L, Wang Z, Sun P et al (2023) TDMO: dynamic multi-dimensional oversampling for exploring data distribution based on extreme gradient boosting learning. *Inf Sci* 649:119621. <https://doi.org/10.1016/j.ins.2023.119621>
21. Sun P, Wang Z, Jia L, Xu Z (2024) SMOTE-kTLNN: a hybrid re-sampling method based on SMOTE and a two-layer nearest neighbor classifier. *Expert Syst Appl* 238:121848. <https://doi.org/10.1016/j.eswa.2023.121848>
22. Khoshgoftaar TM, Rebours P (2007) Improving software quality prediction by noise filtering techniques. *J Comput Sci Technol* 22:387–396. <https://doi.org/10.1007/s11390-007-9054-2>
23. Japkowicz N, Stephen S (2002) The class imbalance problem: a systematic study1. *IDA* 6:429–449. <https://doi.org/10.3233/IDA-2002-6504>
24. Jo T, Japkowicz N (2004) Class imbalances versus small disjuncts. *SIGKDD Explor Newsl* 6:40–49. <https://doi.org/10.1145/1007730.1007737>
25. García V, Sánchez J, Mollineda R (2008) An empirical study of the behavior of classifiers on imbalanced and overlapped data sets. In: Rueda L, Mery D, Kittler J (eds) *Progress in pattern recognition, image analysis and applications*. Springer, Berlin, pp 397–406
26. Napierała K, Stefanowski J, Wilk S (2010) Learning from imbalanced data in presence of noisy and borderline examples. In: Szczuka M, Kryszkiewicz M, Ramanna S et al (eds) *Rough sets and current trends in computing*. Springer, Berlin, pp 158–167
27. Han H, Wang W-Y, Mao B-H (2005) Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In: Huang D-S, Zhang X-P, Huang G-B (eds) *Advances in intelligent computing*. Springer, Berlin, pp 878–887
28. Liu J (2022) Importance-SMOTE: a synthetic minority oversampling method for noisy imbalanced data. *Soft Comput* 26:1141–1163. <https://doi.org/10.1007/s00500-021-06532-4>
29. He H, Bai Y, Garcia EA, Li S (2008) ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence). IEEE, Hong Kong, China, pp 1322–1328
30. Alejo R, García V, Pacheco-Sánchez JH (2015) An efficient over-sampling approach based on mean square error back-propagation for dealing with the multi-class imbalance problem. *Neural Process Lett* 42:603–617. <https://doi.org/10.1007/s11063-014-9376-3>
31. Rivera WA (2017) Noise reduction a priori synthetic over-sampling for class imbalanced data sets. *Inf Sci* 408:146–161. <https://doi.org/10.1016/j.ins.2017.04.046>
32. Bunkhumpornpat C, Sinapiromsaran K, Lursinsap C (2009) Safe-level-: safe-level-synthetic minority over-sampling TEchnique for handling the class imbalanced ProbSMOTElem. In: Theeramunkong T, Kijssirikul B, Cercone N, Ho T-B (eds) *Advances in knowledge discovery and data mining*. Springer, Berlin, pp 475–482
33. Yan YT, Wu ZB, Du XQ et al (2019) A three-way decision ensemble method for imbalanced data oversampling. *Int J Approx Reason* 107:1–16. <https://doi.org/10.1016/j.ijar.2018.12.011>
34. Chen B, Xia S, Chen Z et al (2021) RSMOTE: a self-adaptive robust SMOTE for imbalanced problems with label noise. *Inf Sci* 553:397–428. <https://doi.org/10.1016/j.ins.2020.10.013>
35. Barua S, Islam MdM, Yao X, Murase K (2014) MWMOTE—majority weighted minority oversampling technique for imbalanced data set learning. *IEEE Trans Knowl Data Eng* 26:405–425. <https://doi.org/10.1109/TKDE.2012.232>
36. Ramentol E, Caballero Y, Bello R, Herrera F (2012) SMOTE-RSB \*: a hybrid preprocessing approach based on oversampling and undersampling for high imbalanced data-sets using SMOTE and rough sets theory. *Knowl Inf Syst* 33:245–265. <https://doi.org/10.1007/s10115-011-0465-6>
37. Sáez JA, Luengo J, Stefanowski J, Herrera F (2015) SMOTE–IPF: addressing the noisy and borderline examples problem in imbalanced classification by a re-sampling method with filtering. *Inf Sci* 291:184–203. <https://doi.org/10.1016/j.ins.2014.08.051>



38. Xia S, Wang G, Chen Z et al (2019) Complete random forest based class noise filtering learning for improving the generalizability of classifiers. *IEEE Trans Knowl Data Eng* 31:2063–2078. <https://doi.org/10.1109/TKDE.2018.2873791>
39. Xia S, Chen B, Wang G et al (2022) mCRF and mRD: two classification methods based on a novel multiclass label noise filtering learning framework. *IEEE Trans Neural Netw Learning Syst* 33:2916–2930. <https://doi.org/10.1109/TNNLS.2020.3047046>
40. Zhang A, Yu H, Huan Z et al (2022) SMOTE-RkNN: a hybrid re-sampling method based on SMOTE and reverse k-nearest neighbors. *Inf Sci* 595:70–88. <https://doi.org/10.1016/j.ins.2022.02.038>
41. Kermanidis KL (2009) The effect of borderline examples on language learning. *J Exp Theor Artif Intell* 21:19–42. <https://doi.org/10.1080/09528130802113406>
42. Wilson DL (1972) Asymptotic properties of nearest neighbor rules using edited data. *IEEE Trans Syst Man, Cybern SMC-2*:408–421. <https://doi.org/10.1109/TSMC.1972.4309137>
43. Brodley CE, Friedl MA (1999) Identifying mislabeled training data. *Jair* 11:131–167. <https://doi.org/10.1613/jair.606>
44. Verbaeten S, Van Assche A (2003) Ensemble methods for noise elimination in classification problems. In: Windeatt T, Roli F (eds) *Multiple classifier systems*. Springer, Berlin, pp 317–325
45. Stefanowski J (2013) Overlapping, rare examples and class decomposition in learning classifiers from imbalanced data. In: Ramanna S, Jain LC, Howlett RJ (eds) *Emerging paradigms in machine learning*. Springer, Berlin, pp 277–306
46. Islam A, Belhaouari SB, Rehman AU, Bensmail H (2022) KNNOR: an oversampling technique for imbalanced datasets. *Appl Soft Comput* 115:108288. <https://doi.org/10.1016/j.asoc.2021.108288>
47. Kunakorntrum I, Hinthong W, Phunchongharn P (2020) A synthetic minority based on probabilistic distribution (SyMProD) oversampling for imbalanced datasets. *IEEE Access* 8:114692–114704. <https://doi.org/10.1109/ACCESS.2020.3003346>
48. Wang AX, Chukova SS, Nguyen BP (2023) Synthetic minority oversampling using edited displacement-based k-nearest neighbors. *Appl Soft Comput* 148:110895. <https://doi.org/10.1016/j.asoc.2023.110895>
49. Zeraatkar S, Afsari F (2021) Interval-valued fuzzy and intuitionistic fuzzy-KNN for imbalanced data classification. *Expert Syst Appl* 184:115510. <https://doi.org/10.1016/j.eswa.2021.115510>
50. Bradley AP (1997) The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recognit* 30:1145–1159. [https://doi.org/10.1016/S0031-3203\(96\)00142-2](https://doi.org/10.1016/S0031-3203(96)00142-2)
51. Li T, Wang Y, Liu L et al (2023) Subspace-based minority oversampling for imbalance classification. *Inf Sci* 621:371–388. <https://doi.org/10.1016/j.ins.2022.11.108>
52. Shi S, Xiong H, Li G (2023) A no-tardiness job shop scheduling problem with overtime consideration and the solution approaches. *Comput Ind Eng* 178:109115. <https://doi.org/10.1016/j.cie.2023.109115>

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

# Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

[onlineservice@springernature.com](mailto:onlineservice@springernature.com)